

A BEGINNER'S GUIDE TO

ticket-mng

How a real, production-style backend is put together,
explained one file at a time

Node.js · TypeScript · Fastify · PostgreSQL · Redis · BullMQ · WebSockets · Docker

Contents

1. [How to use this guide](#)
2. [What the app does, in plain words](#)
3. [Words you need first](#)
4. [The tools \(the stack\) and why each one is there](#)
5. [A tour of the folders](#)
6. [How the program starts](#)
7. [The database: tables, migrations, queries, transactions](#)
8. [The journey of one request](#)
9. [Validation and errors](#)
10. [Authentication: who are you?](#)
11. [Venues and events](#)
12. [Bookings: the hardest part](#)
13. [Background jobs and the outbox](#)
14. [Payments and webhooks](#)
15. [Tickets and check-in](#)
16. [Posters: uploading files](#)
17. [Live seat maps \(WebSockets\)](#)
18. [The browser page \(public/app.js\)](#)
19. [Staying fast under load](#)
20. [Logs, metrics, health checks, shutdown](#)
21. [Docker, Nginx, CI and deploying](#)
22. [Tests](#)
23. [Trace one purchase from start to finish](#)
24. [How to build your own project](#)
25. [Glossary](#)

How to use this guide

You said you know programming languages but have never built a big project. That is exactly the gap this guide fills. Knowing a language means you can write a function. Building a project means knowing **how many pieces connect**: a web server, a database, a cache, background workers, a browser page, and the glue between them.

This guide walks through **ticket-mng**, an event-ticketing backend (think a mini Ticketmaster). It is a very good project to learn from because it solves real, hard problems: two people clicking the same seat at the same millisecond, payments that arrive late, emails that must be sent exactly once, thousands of people watching one seat map.

How it is organised

- Chapters 2 to 5 give you the **big picture**: what the app does, the vocabulary, the tools, the folders.
- Chapters 6 to 9 explain the **skeleton** every backend has: startup, database, request handling, errors.
- Chapters 10 to 18 go **feature by feature**, showing the real code with line-by-line explanations.
- Chapters 19 to 22 cover **running it for real**: speed, monitoring, Docker, tests.
- Chapters 23 and 24 tie it together and show you **how to start your own project**.

How to read the code boxes. Code is copied from the repository (sometimes shortened with `// ...`). Right after each box you will find an explanation. File paths are written like `src/app.ts`, so you can open the same file and follow along.

Tip. Keep the project open in your editor while you read. When the guide says "open `src/db/transaction.ts`", actually open it. Reading real files is how this knowledge sticks.

You do not need to understand everything on the first read. Some chapters (bookings, payments) are hard even for experienced developers. Read for the idea first; come back for the details later.

What the app does, in plain words

Imagine a concert hall. The app lets people:

1. **Sign up and log in** (as an *attendee* who buys tickets, or an *organizer* who creates events; there is also an *admin*).
2. An organizer **creates a venue** ("City Hall, 3 sections, 20 rows each, 30 seats per row"). The app generates every seat with a row letter, number and x/y position for drawing.
3. The organizer **creates an event** at that venue ("Rock Night, Friday 8pm"), sets a price per section and **publishes** it.
4. A buyer **opens the seat map**, sees which seats are free, and **holds** some seats. A hold reserves the seats for 10 minutes.
5. The buyer **pays** (with a fake card in development, or Stripe). When the payment succeeds, the booking becomes **confirmed**.
6. The buyer gets an **email with QR-code tickets**.
7. At the door, staff **scan the QR code** (check-in). Each ticket admits exactly once.
8. Meanwhile, everyone else watching the seat map **sees seats turn red live**, without refreshing.

Behind that simple story are the hard problems the project is really about:

Problem	Where it is solved
200 people click the same seat at once; exactly one must win	Chapter 12 (bookings)
A hold must expire after 10 minutes even if a server crashes	Chapter 12 (lazy expiry)
An email must be sent once, and only if the booking really saved	Chapter 13 (outbox)
Payment messages arrive twice, late, or out of order	Chapter 14 (reconcile)
Thousands watch one seat map	Chapters 17 and 19
Deploying new code must not drop a single request	Chapter 21

Words you need first

If any of these are new, read this chapter slowly. Everything later builds on them.

Client and server. The *client* asks, the *server* answers. Your browser is a client. `ticket-mng` is a server. A phone app, a script, or another server can also be a client.

HTTP. The language clients and servers use on the web. A request has a **method** (what to do), a **path** (what to do it to), **headers** (extra info, like who you are) and sometimes a **body** (data). The response has a **status code** and usually a body.

Method	Meaning	Example in this app
GET	read something	GET <code>/api/v1/events</code> lists events
POST	create / do something	POST <code>/api/v1/auth/login</code> logs in
PATCH	change part of something	PATCH <code>/api/v1/events/:id</code> edits an event
DELETE	remove something	DELETE <code>/api/v1/events/:id</code> deletes a draft

Status codes you will see: `200` OK, `201` Created, `204` No Content, `400` bad input, `401` not logged in, `403` logged in but not allowed, `404` not found, `409` conflict (for example the seat is taken), `422` input makes no sense, `429` too many requests, `500` server bug, `503` server busy, try again.

API. "Application Programming Interface": the list of requests a server accepts. This app's API is all under `/api/v1/...` (the `v1` means "version 1", so a future `v2` can change things without breaking old clients).

JSON. The text format for data: `{"email": "a@b.com", "seatIds": [12, 13]}`. Almost every request and response body in this app is JSON.

Route / endpoint. One method + path the server handles, like `POST /api/v1/events/:id/bookings`. `:id` is a *parameter*: a placeholder filled by the real id.

Handler. The function that runs for a route.

Database. Where data is stored permanently. This app uses **PostgreSQL**, a *relational* database: data lives in **tables** (like spreadsheets) with **rows** and **columns**. You talk to it using **SQL** (`SELECT`, `INSERT`, `UPDATE`, `DELETE`).

Transaction. A group of database changes that happen *all together or not at all*. If anything fails halfway, the database undoes (rolls back) everything. Saving money-related data without transactions is how bugs lose money.

Cache. A fast, temporary copy of data so you do not have to recompute or re-query it every time. This app uses **Redis** (an in-memory data store) and plain in-memory JavaScript maps as caches.

Queue / job / worker. Some work is slow (sending email, resizing an image) or must happen later (expire a hold in 10 minutes). Instead of doing it during the request, the app puts a *job* on a *queue*, and a separate *worker* process picks it up.

Process. A running program. This project runs *several* processes from the *same code*: the **API server** (answers HTTP), the **worker** (runs jobs), and a **migration** command (updates the database structure).

Concurrency / race condition. Many things happening at the same time. A *race condition* is a bug that happens only when two things interleave in an unlucky order, for example two requests both reading "seat is free" before either writes "seat is taken".

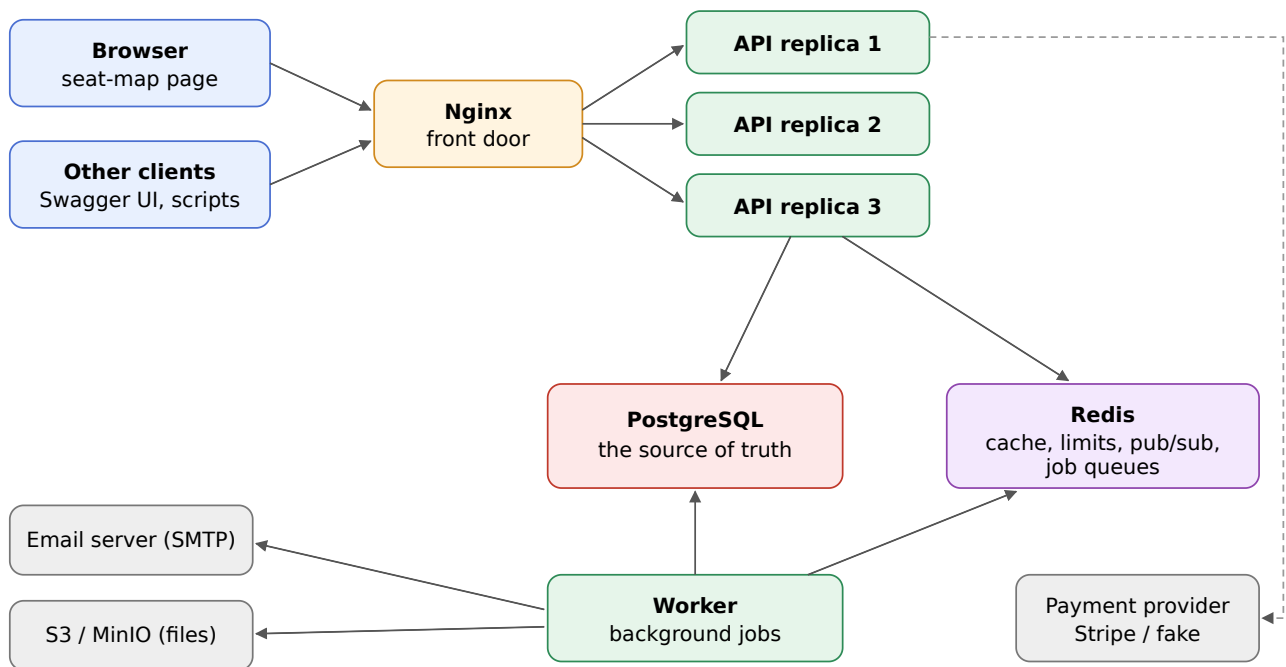
Environment variables. Settings passed to a program from outside (like `DATABASE_URL=postgres://...`), so the same code runs on your laptop and in production with different settings.

Container (Docker). A packaged, isolated box containing your program and everything it needs, so it runs the same everywhere.

WebSocket. A connection that stays open so the server can *push* messages to the browser (used for live seat updates). Normal HTTP is "ask, get one answer, done".

The tools (the stack) and why each one is there

A "stack" is the set of tools a project is built with. Beginners often think "a backend = one program". In reality, a backend is **your program plus several services it talks to**. Here is the whole system as a picture:



The whole system. Arrows mean "talks to". Everything in green is our code (the same codebase run in different ways).

Now each tool, what it is, and **why this project uses it**.

Node.js (version 24). A program that runs JavaScript outside the browser. Node is good at servers because it handles thousands of connections at once using an *event loop*: instead of waiting (blocking) while the database answers, it moves on to other requests and comes back when the answer arrives. That is why you see `async / await` everywhere in the code.

TypeScript. JavaScript plus *types*. You write `function f(id: string): Promise<Booking>` and the compiler checks, before the program runs, that you never pass a number where a string is expected. In a big project this catches a huge number of mistakes. TypeScript is compiled (`npm run build`) into plain JavaScript in the `dist/` folder, and that is what runs in production. In development, `tsx` runs `.ts` files directly.

Fastify. The web framework: it listens for HTTP requests and calls your handler for each route. It is like Express but faster and with built-in support for schemas (descriptions of what the input and output look like). It has **hooks** (functions that run at stages of every request, like "on request" and "on response") and **plugins** (a way to group routes and settings).

Zod. A library to describe the *shape* of data: `z.object({ email: z.email(), password: z.string().min(8) })`. In this project, one Zod schema per route does three jobs at once: it **validates** incoming data (bad input gets a `400` automatically), gives TypeScript the **types**, and generates the **API documentation** you see at `/docs`.

PostgreSQL. The database. It is the **single source of truth**: who owns which seat, what was paid, which ticket is valid. It also *enforces rules itself* (constraints), so even buggy code can't break them.

Kysely. A "query builder": you write database queries in TypeScript, like `db.selectFrom('users').where('id', '=', id)`, and it produces SQL. Unlike an ORM (such as Prisma), it doesn't hide the SQL, which matters because this project needs precise control over locking.

Redis. A very fast in-memory store. Here it is used for: caches, rate limits, the "claim gate" (a fast pre-check before booking seats), a denylist of logged-out sessions, pub/sub messages for live seat updates, and the job queues. Important rule of this project: **Postgres decides, Redis accelerates**. If Redis loses its data, nothing important is lost.

BullMQ. A job-queue library built on Redis. The API adds jobs; the worker process takes them and runs them, with retries.

WebSockets. Long-lived connections so the server can push seat updates to browsers instantly.

Nginx. A web server placed *in front of* the app (a "reverse proxy"). It handles HTTPS encryption, spreads requests across the 3 API copies (load balancing), and applies a first rate limit.

Docker and Docker Compose. Docker packages the app into an *image*. Compose starts many containers together (Postgres, Redis, 3 API copies, worker, Nginx...) with one command: `docker compose up`.

Others. `pino` (logging), `prom-client` (metrics for Prometheus), `jose` (JWT tokens), `@node-rs/argon2` (password hashing), `sharp` (image resizing), `nodemailer` (email), `qrcode` (QR images), `@aws-sdk/client-s3` (file storage), `stripe` (payments), `vitest` (tests), `eslint/prettier` (code style), `k6` (load testing), `playwright` (browser tests).

Lesson for your own projects. You do not need all of these to start. A first project needs: a language + a web framework + a database. Add Redis, queues, WebSockets, Docker only when you have the problem they solve. This project added them in phases (see the roadmap in `README.md`), which is exactly how you should grow a project too.

A tour of the folders

ticket-mng/	
├─ package.json	project name, dependencies, and the npm scripts (commands)
├─ tsconfig.json	TypeScript compiler settings
├─ .env.example	example settings; copy to .env for local development
├─ Dockerfile	how to build the Docker image
├─ docker-compose.yml	the whole system on one machine
├─ src/	THE APPLICATION CODE
│ ├─ server.ts	entry point #1: starts the HTTP API
│ ├─ worker.ts	entry point #2: starts the background-job worker
│ ├─ app.ts	builds the Fastify app: hooks, plugins, routes
│ ├─ config.ts	reads and validates environment variables
│ ├─ db/	database connection, transactions, migrations, table types
│ ├─ lib/	shared helpers: errors, logging, caching, rate limits, ...
│ ├─ modules/	one folder per feature (auth, users, venues, events, ...)
│ ├─ jobs/	queues, the outbox relay, job runner, job handlers, schedules
│ ├─ realtime/	live seat-map updates (WebSockets + Redis pub/sub)
│ ├─ emails/	email templates
│ └─ fake-gateway/	a pretend payment provider for offline development
├─ public/	the browser page (index.html, app.js, styles.css)
├─ test/	automated tests (unit + API)
├─ scripts/	seed data, race test, load tests, checks
├─ deploy/	production configs: Nginx, monitoring, rollout script
└─ docs/	architecture notes, decision records (ADRs), study guide

The most important organising idea: "modules" by feature. Look inside `src/modules/`:

modules/	
├─ auth/	signup, login, tokens, sessions, password reset
├─ users/	profiles and roles
├─ venues/	venues and generating seat layouts
├─ events/	events, search, seat maps, posters
├─ bookings/	holding seats, cancelling, expiring (the hard part)
├─ payments/	paying, webhooks, refunds
├─ tickets/	QR tickets and check-in
├─ admin/	queue inspection, invariant checks
└─ health/	"am I alive / ready?" endpoints

Each module usually has:

- `routes.ts` : the HTTP endpoints (URL, input schema, output schema, and a short handler);
- `service.ts` : the business logic (the actual rules), independent of HTTP;
- sometimes `schemas.ts` / `dto.ts` (data shapes) and helpers.

Why split routes from services? The route deals with HTTP things (headers, status codes, who is logged in). The service deals with business rules ("a seat can only be held if free"). This way the same service can be called from an HTTP route, from a background job, or from a test, without HTTP being involved. The booking service, for example, is called by the booking route *and* by the worker's expiry job.

`src/lib/` **is the toolbox.** Things used by many modules: `errors.ts` (one error format), `logger.ts`, `metrics.ts`, `cache.ts`, `rate-limit.ts`, `idempotency.ts`, `pagination.ts`, `redis.ts`, `storage.ts` (file uploads), `mailer.ts` (email), and so on.

How the program starts

Step 0: npm scripts

`package.json` lists the commands you run. The important ones:

```
"scripts": {
  "dev": "tsx watch src/server.ts",
  "worker": "tsx watch src/worker.ts",
  "migrate": "tsx src/db/migrate.ts latest",
  "seed": "tsx scripts/seed.ts",
  "test": "vitest run",
  "build": "tsc -p tsconfig.build.json",
  "start": "node dist/server.js",
  "start:worker": "node dist/worker.js"
}
```

- `npm run dev` runs `src/server.ts` with `tsx` in *watch mode* (it restarts when you save a file).
- `npm run worker` does the same for the worker.
- `npm run migrate` creates/updates the database tables.
- `npm run build` compiles TypeScript to JavaScript in `dist/`; `npm start` runs that compiled code (what production does).

So in development you open **two terminals**: one with `npm run dev` (API), one with `npm run worker` (jobs). Both need Postgres and Redis running.

Step 1: configuration (`src/config.ts`)

The very first thing that happens is reading settings. Here is the pattern, shortened:

```
import { z } from 'zod';

// Load .env if present. Real environment variables win over the file.
try {
  process.loadEnvFile();
} catch {
  // no .env file, which is fine in CI/production
}

const schema = z.object({
  NODE_ENV: z.enum(['development', 'test', 'production']).default('development'),
  PORT: z.coerce.number().int().positive().default(3000),
  DATABASE_URL: z.string().min(1),
  JWT_ACCESS_SECRET: z.string().min(32, 'JWT_ACCESS_SECRET must be at least 32 characters'),
  HOLD_TTL_SECONDS: z.coerce.number().int().min(30).max(3600).default(600),
  // ... about 50 settings in total
});

const parsed = schema.safeParse(process.env);
if (!parsed.success) {
  console.error('Invalid environment configuration:\n' + z.prettifyError(parsed.error));
  process.exit(1);
}
```

```

}
// ... extra production safety checks ...
export const config = parsed.data;

```

Line by line:

- `process.loadEnvFile()` reads the `.env` file (a list of `KEY=value` lines) into `process.env`. It's wrapped in `try` because in production there is no `.env` file; settings come from the real environment.
- `schema` describes **every setting**: its type, limits and default. `z.coerce.number()` converts the text `"3000"` into the number `3000` (environment variables are always text).
- `schema.safeParse(process.env)` checks all settings at once. If anything is wrong (a missing `DATABASE_URL`, a secret that's too short), the program prints a clear message and **stops immediately** with `process.exit(1)`.
- `export const config = parsed.data;` Every other file imports `config` and gets typed, validated values: `config.PORT` is guaranteed to be a positive integer.

Below the schema are **production guards**: the app refuses to start in production with the development secrets from `.env.example`, with the fake payment gateway, or with the deliberately broken `naive` booking strategy.

```

if (parsed.data.NODE_ENV === 'production' && parsed.data.PAYMENT_PROVIDER === 'fake') {
  console.error('The fake payment gateway accepts any test card; it is not allowed in production');
  process.exit(1);
}

```

Lesson: fail fast at startup. If a setting is wrong, it is far better to crash immediately with a clear message than to start "successfully" and fail an hour later when the first user tries to pay. Copy this pattern into every project.

Step 2: the entry point (`src/server.ts`)

```

const app = await buildApp({ loggerInstance: logger });

async function closeResources() {
  await closeQueues();
  await Promise.allSettled([db.destroy(), redis.quit()]);
}

let stopping = false;
for (const signal of ['SIGINT', 'SIGTERM'] as const) {
  process.once(signal, () => {
    if (stopping) return;
    stopping = true;
    lifecycle.beginShutdown();
    void (async () => {
      if (config.SHUTDOWN_DRAIN_MS) await sleep(config.SHUTDOWN_DRAIN_MS);
      await app.close();
      await closeResources();
      process.exit(0);
    })();
  });
});

```

```

}

try {
  await app.listen({ host: config.HOST, port: config.PORT });
} catch (err) {
  app.log.error(err);
  await closeResources();
  process.exit(1);
}

```

What it does:

1. `buildApp(...)` builds the whole web application (next section).
2. It registers **signal handlers**. When you press Ctrl+C (`SIGINT`) or Docker stops the container (`SIGTERM`), instead of dying instantly the server shuts down *gracefully*: it marks itself "not ready" (so the load balancer stops sending traffic), optionally waits a bit (`SHUTDOWN_DRAIN_MS`), stops accepting new connections, lets in-flight requests finish, and closes the database and Redis connections. This is why deploys don't drop requests (chapter 20).
3. `app.listen(...)` opens the port (3000 by default). From now on, the server answers requests.

Note the top-level `await` : this file is an ES module (`"type": "module"` in `package.json`), so it can use `await` outside a function.

Step 3: building the app (`src/app.ts`)

`buildApp` is the "assembly line" of the server. It is long, so here are its parts in order:

```

export async function buildApp(opts: FastifyServerOptions = {}, overrides: AppOverrides = {}) {
  const app = Fastify({
    genReqId, // how to name each request
    logController: new LogController({ requestIdLogLevel: 'requestId' }),
    ...opts,
    trustProxy, // believe X-Forwarded-For only from Nginx
  }).withTypeProvider<ZodTypeProvider>();
}

```

- `Fastify({...})` creates the server object.
- `genReqId` gives each request an **id**. If Nginx already sent an `X-Request-Id` header (and it looks safe), that id is reused, otherwise a random UUID is generated. Every log line of that request includes this id, so you can find all lines of one request.
- `.withTypeProvider<ZodTypeProvider>()` tells TypeScript "route schemas are Zod schemas", so inside a handler `req.body` has the right type automatically.

Then **hooks** are added. A hook is a function Fastify runs at a certain moment of *every* request:

```

app.addHook('onRequest', async () => {
  httpRequestsInFlight.inc(); // metrics: one more request running
});
app.addHook('onResponse', async (req, reply) => {
  httpRequestsInFlight.dec();
  httpRequestDuration.observe({ method: req.method, route: req.routeOptions.url ??
'unmatched',

```

```

    status_code: String(reply.statusCode) }, reply.elapsedTime / 1000);
  });

  // Make the request id available everywhere without passing it around
  app.addHook('onRequest', (req, _reply, done) => requestContext.run({ requestId: req.id },
done));

  // Per-IP rate limit for the whole API (shared across servers via Redis)
  app.addHook('onRequest', async (req, reply) => {
    if (!req.url.startsWith('/api/') || req.url.startsWith('/api/v1/webhooks/')) return;
    await enforce(req, reply, [[rule, req.ip]]);
  });

  // Add x-served-by and x-request-id headers to every response
  app.addHook('onSend', async (req, reply) => {
    reply.header('x-served-by', INSTANCE_ID).header('x-request-id', req.id);
  });

```

Then **plugins** and settings:

```

const hub = new LiveSeatHub(); // live seat updates (chapter 17)
app.decorate('liveHub', hub);
app.addHook('preClose', async () => hub.close());
await app.register(websocket, { options: { maxPayload: 4096 } });

app.setValidatorCompiler(validatorCompiler); // use Zod to validate input
app.setSerializerCompiler(serializerCompiler); // use Zod to shape output

await app.register(cookie); // read/write cookies
app.decorateRequest('user', null); // every request gets req.user (null =
anonymous)

await app.register(swagger, { openapi: { /* title, tags, security */ } });
await app.register(swaggerUi, { routePrefix: '/docs' }); // interactive docs at /docs

app.setErrorHandler(errorHandler); // one place turns errors into JSON (chapter 9)
app.setNotFoundHandler(/* JSON 404 */);

await app.register(fastifyStatic, { root: PUBLIC_DIR, /* security headers */ }); // serves
public/

```

- `app.decorate('name', value)` attaches something to the app object so any route can use it (`app.liveHub`).
- `app.register(plugin)` plugs in a feature. `@fastify(cookie)` adds cookie support, `@fastify/swagger` builds API docs, `@fastify/static` serves files from `public/` (the seat-map page).

Finally, **routes**:

```

await app.register(healthRoutes);
if (config.PAYMENT_PROVIDER === 'fake') await app.register(fakeGatewayRoutes);
await app.register(
  async (api) => {
    await api.register(authRoutes);
    await api.register(userRoutes);
    await api.register(venueRoutes);
    await api.register(eventRoutes);
    await api.register(posterRoutes);
    await api.register(bookingRoutes);
  }
);

```

```

    await api.register(paymentRoutes);
    await api.register(ticketRoutes);
    await api.register(adminRoutes);
  },
  { prefix: '/api/v1' },
);
return app;
}

```

Each `xxxRoutes` is a plugin defined in `src/modules/xxx/routes.ts`. Registering them inside a plugin with `prefix: '/api/v1'` means a route written as `/auth/login` in `auth/routes.ts` is actually served at `/api/v1/auth/login`.

Why `buildApp` is a function instead of code that runs at import. Tests call `buildApp()` to get a fresh app without opening a real port (`app.inject(...)` sends fake requests). The race-test script calls it with different booking strategies. Making your app "buildable" by a function is a great habit.

Step 4: the worker entry point (`src/worker.ts`)

The worker is the same codebase with a different starting file. It does not answer HTTP API calls; it:

1. creates one BullMQ `Worker` per queue (`email`, `bookings`, `payments`, `media`, `maintenance`) with a *concurrency* (how many jobs of that queue run at once);
2. starts the **outbox relay** (chapter 13), which moves jobs from the database into Redis;
3. registers **recurring schedules** (every 30 s, every 15 min, daily at 03:00);
4. starts a tiny HTTP server only for health checks and metrics (port 3100);
5. handles `SIGTERM` gracefully (finish running jobs, then exit).

```

const workers = [
  createWorker('email', handlers.email, 10),           // I/O bound: many in parallel
  createWorker('bookings', handlers.bookings, 20),
  createWorker('payments', handlers.payments, 10),
  createWorker('media', handlers.media, 2),           // CPU heavy (image resizing): few
  createWorker('maintenance', handlers.maintenance, 1),
];
const relay = new OutboxRelay();
await relay.start();
await registerSchedules();

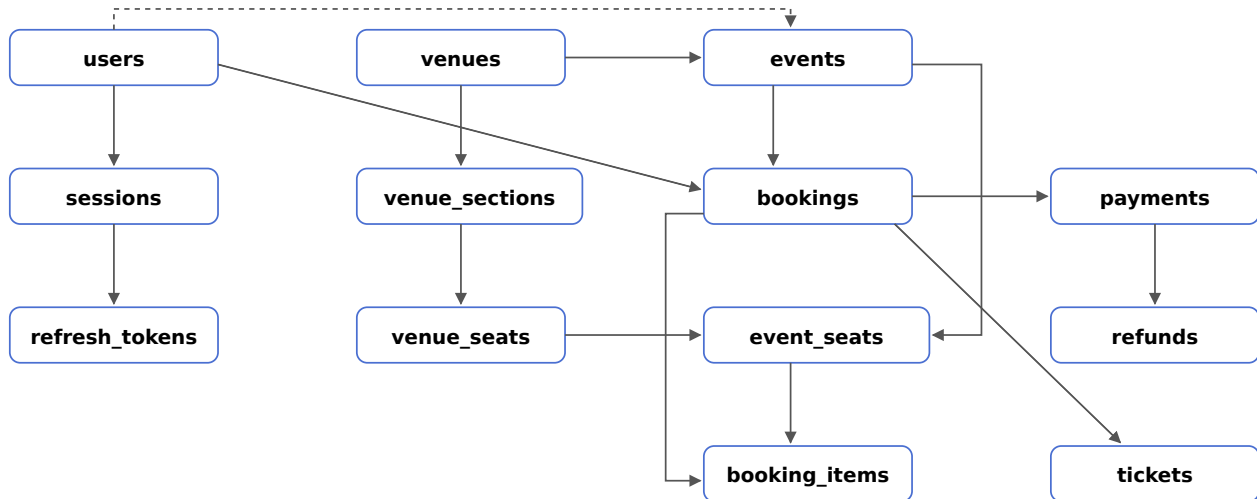
```

Why two processes? If sending an email took 2 seconds inside the request, the buyer would wait 2 seconds, and if the email server were down, the purchase would fail. Moving slow or retryable work into a separate worker keeps requests fast and reliable, and lets you scale each part separately (more workers for a big email backlog, more API copies for traffic).

The database: tables, migrations, queries, transactions

The database is the heart of this project. If you understand the tables, you understand what the app *is*.

The tables and how they relate



Arrow A → B means "one A has many B" (B stores A's id as a foreign key).

Also: password_reset_tokens, notifications (sent emails), outbox (jobs), webhook_events, idempotency_keys.
event_seats.booking_id points back at the booking currently holding / owning that seat.

The data model (simplified).

In words:

- **users** have many **sessions** (one per login/device); each session has **refresh_tokens** (chapter 10).
- A **venue** has **venue_sections**, which have **venue_seats**. This is the *physical* building: seat "Balcony, row C, seat 12" at grid position x=11, y=24. It never changes.
- An **event** happens at a venue and is organised by a user.
- **event_seats** has **one row per seat per event**, copied from `venue_seats` when the event is created, with its own **price**, **status** (`available`, `held`, `booked`), **booking_id** (who holds it) and **version** (a counter bumped on every change).
- A **booking** belongs to a user and an event, has a status (`pending`, `confirmed`, `expired`, `cancelled`, `refunded`) and an `expires_at` time. Its seats are listed in **booking_items**.
- A booking has **payments**; a payment can have **refunds**; a confirmed booking has one **ticket** per seat.

Why copy venue seats into event_seats? The venue is a template. Each event sells its own inventory with its own prices. Booking a seat for Friday's concert must not touch Saturday's. Separating "the template" from "the stock" is a pattern you will reuse (products vs. inventory, courses vs. enrolments, and so on).

Migrations: how tables are created

You never create tables by hand. You write **migrations**: numbered files that change the database structure step by step. They live in `src/db/migrations/`:

```
0001_initial_schema.ts      users, venues, seats, events, event_seats
0002_auth.ts                passwords, sessions, refresh tokens, reset tokens
0003_bookings.ts            bookings, booking_items, seat holders
0004_jobs_tickets_posters.ts outbox, notifications, tickets, poster columns
0005_payments.ts            payments, refunds, webhook_events, idempotency_keys
0006_outbox_notify_immediate_only.ts a performance fix found by load testing
```

Each file has an `up` function (apply the change) and a `down` function (undo it). Kysely's migrator remembers which ones already ran (in its own `kysely_migration` table), so `npm run migrate` only applies new ones. The rule written at the top of `0001`: **never edit a migration that has already run anywhere; add a new one instead**. That's how every developer's database, the test database and production end up with the same structure.

Here is part of `0001_initial_schema.ts`. The SQL is written by hand on purpose:

```
CREATE TYPE event_status AS ENUM ('draft', 'published', 'cancelled');

CREATE TABLE events (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  organizer_id uuid NOT NULL REFERENCES users (id) ON DELETE RESTRICT,
  venue_id    uuid NOT NULL REFERENCES venues (id) ON DELETE RESTRICT,
  title       text NOT NULL CHECK (char_length(title) BETWEEN 1 AND 200),
  status      event_status NOT NULL DEFAULT 'draft',
  starts_at   timestampz NOT NULL,
  ends_at     timestampz NOT NULL,
  search      tsvector GENERATED ALWAYS AS (
    setweight(to_tsvector('english', title), 'A') ||
    setweight(to_tsvector('english', description), 'B')
  ) STORED,
  CHECK (ends_at > starts_at),
  CONSTRAINT events_no_venue_overlap EXCLUDE USING gist (
    venue_id WITH =,
    tstzrange(starts_at, ends_at) WITH &&
  ) WHERE (status <> 'cancelled')
);
CREATE INDEX events_listing_idx ON events (status, starts_at, id);
CREATE INDEX events_search_idx ON events USING gin (search);
```

Piece by piece:

- `CREATE TYPE ... AS ENUM` makes a type that only allows those exact values. The database will reject `status = 'publishd'` (typo).

- `uuid PRIMARY KEY DEFAULT gen_random_uuid()` : every event gets a random unique id automatically.
- `REFERENCES users (id)` is a **foreign key**: `organizer_id` must be an id that exists in `users`. `ON DELETE RESTRICT` means you cannot delete a user who organises events.
- `CHECK (...)` : a rule checked on every insert/update. An event can't end before it starts, whatever the code does.
- `search ... GENERATED ALWAYS AS (...) STORED` : a column Postgres computes itself from title and description, used for full-text search ("rock concert" finds "Rock Night Concert").
- `EXCLUDE USING gist (...)` : **no two non-cancelled events at the same venue may overlap in time.** `&&` means "ranges overlap". This is enforced by the database even when two organisers submit at the exact same moment. A check in application code ("SELECT any overlapping event, then INSERT") would have a race: both could check before either inserts.
- `CREATE INDEX ...` : an index is like a book's index; it makes lookups fast. `events_listing_idx` exactly matches the main listing query "published events, ordered by start time".

The table below lists the rules the database enforces. Learning to push rules into the database is one of the most valuable lessons of this project:

Rule	Enforced by
No two events overlap at a venue	<code>EXCLUDE</code> constraint (above)
A seat is <code>available</code> exactly when no booking points at it	<code>CHECK ((status = 'available') = (booking_id IS NULL))</code>
One active hold per user per event	partial unique index on <code>bookings (user_id, event_id)</code> <code>WHERE status = 'pending'</code>
One valid ticket per seat	partial unique index on <code>tickets (event_seat_id)</code> <code>WHERE status = 'valid'</code>
A webhook is processed once	primary key <code>(provider, event_id)</code> on <code>webhook_events</code>
At most one live refund per payment	partial unique index on <code>refunds</code>

A **partial unique index** is a uniqueness rule that only applies to some rows. "Unique `(user_id, event_id)` *where* status is pending" means a user can have many old bookings for an event, but only one pending hold at a time.

Table types for TypeScript (`src/db/types.ts`)

Kysely needs to know the shape of each table to type-check queries. `types.ts` describes them in TypeScript:

```
export const USER_ROLES = ['attendee', 'organizer', 'admin'] as const;
export type UserRole = (typeof USER_ROLES)[number]; // 'attendee' | 'organizer' | 'admin'

export interface UsersTable {
  id: Generated<string>; // the database generates it; optional on insert
  email: string;
```

```

name: string;
role: Generated<UserRole>;           // has a default
passwordHash: string | null;
createdAt: CreatedAt;                // set by the database, never by us
updatedAt: UpdatedAt;                // maintained by a trigger
}

```

Notice `passwordHash` in camelCase while the column is `password_hash`. The `CamelCasePlugin` (next section) translates automatically. The `as const` array trick gives you both a list you can loop over (for Zod enums) and a type.

Connecting: the pool (`src/db/index.ts`)

```

pg.types.setTypeParser(pg.types.builtins.INT8, (v) => Number(v));

export const pool = new pg.Pool({
  connectionString: config.DATABASE_URL,
  max: config.DB_POOL_MAX,                // at most N connections at once
  connectionTimeoutMillis: config.DB_CONNECT_TIMEOUT_MS, // wait max 5 s for a free one
  idleTimeoutMillis: 30_000,
  statement_timeout: config.DB_STATEMENT_TIMEOUT_MS,    // Postgres kills queries > 15 s
  idle_in_transaction_session_timeout: 30_000,          // and forgotten transactions
  application_name: 'ticket-mng',
});

export const db = new Kysely<DB>({
  dialect: new PostgresDialect({ pool }),
  plugins: [new CamelCasePlugin()],
});

```

- Opening a database connection is slow, so apps keep a **pool** of open connections and lend them out. `max` limits how many run at once. If all are busy, a request waits; if it waits longer than `connectionTimeoutMillis`, it fails, and the error handler turns that into `503 SERVICE_BUSY` ("try again shortly") instead of hanging forever.
- The timeouts are **safety nets**: a runaway query or a transaction someone forgot to finish (holding locks!) is cut off by Postgres itself.
- `db` is the object every file imports to run queries.
- The first line: Postgres big integers (`bigint`, `count(*)`) come back as text by default; this converts them to JavaScript numbers.

Writing queries with Kysely

Kysely reads almost like SQL:

```

// SELECT id, email, name, role, created_at FROM users WHERE id = $1
const user = await db
  .selectFrom('users')
  .select(['id', 'email', 'name', 'role', 'createdAt'])
  .where('id', '=', currentUser(req).id)
  .executeTakeFirst();           // one row or undefined

```

```

// UPDATE users SET name = $1 WHERE id = $2 RETURNING ...
const user = await db

```

```

.updateTable('users')
.set({ name: req.body.name })
.where('id', '=', currentUser(req).id)
.returning(userColumns)
.executeTakeFirst();

```

The values (\$1 , \$2) are sent separately from the SQL text, which prevents **SQL injection** (a user typing ' ; DROP TABLE users; -- as their name does nothing bad). When a query is clearer in raw SQL, the code uses the `sql` tag, which is still safe because `${...}` values become parameters:

```

await sql`
INSERT INTO venue_seats (section_id, row_label, seat_number, x, y)
SELECT * FROM unnest(
  ${seats.map((s) => sectionId.get(s.section))}::uuid[],
  ${seats.map((s) => s.rowLabel)}::text[],
  ${seats.map((s) => s.seatNumber)}::int[],
  ${seats.map((s) => s.x)}::int[],
  ${seats.map((s) => s.y)}::int[]
)
`.execute(trx);

```

This inserts thousands of seats in **one** statement: it sends five arrays and `unnest` turns them into rows. Inserting 50,000 seats one by one would mean 50,000 network round trips.

Transactions (`src/db/transaction.ts`)

A transaction groups changes so they all happen or none do. Kysely's basic form:

```

const venue = await db.transaction().execute(async (trx) => {
  const venue = await
  trx.insertInto('venues').values({...}).returningAll().executeTakeFirstOrThrow();
  await trx.insertInto('venueSections').values(...).execute();
  await sql`INSERT INTO venue_seats ...`.execute(trx);
  return venue;
});

```

Inside the callback you use `trx` instead of `db`. If any line throws, everything is rolled back: you never get a venue with half its seats.

This project wraps it in a smarter helper, `withTransaction`:

```

const RETRYABLE = new Set([
  '40001', // serialization_failure
  '40P01', // deadlock_detected
]);

export async function withTransaction<T>(<
  fn: (trx: Transaction<DB>) => Promise<T>,
  { isolation = 'read committed', retries = 3 } : TransactionOptions = {},
): Promise<T> {
  for (let attempt = 0; ; attempt++) {
    const effects: Effect[] = [];
    try {
      const result = await pendingEffects.run(effects, () =>
        db.transaction().setIsolationLevel(isolation).execute(fn),

```

```

    );
    await runEffects(effects);          // run "after commit" effects
    return result;
  } catch (err) {
    if (!isRetryable(err) || attempt >= retries) throw err;
    await sleep(Math.random() * 10 * 2 ** attempt); // jittered backoff, then retry
  }
}
}

```

Two extra features:

1. **Automatic retry.** Under heavy concurrency, Postgres sometimes aborts a transaction on purpose, with a *deadlock* (two transactions waiting for each other) or a *serialization failure* (in the strictest isolation mode, it detected that two transactions conflicted). Nothing is wrong with the request; running it again works. So the helper retries up to 3 times, waiting a small random time (*jitter*) so retries don't collide again.
2. **After-commit effects.** Some things must happen only *after* the data is really saved: telling live seat maps "this seat is now held", or clearing caches. If you did them inside the transaction and it then rolled back, viewers would see a seat change that never happened.

```

export function afterCommit(effect: Effect): void {
  const effects = pendingEffects.getStore();
  if (effects) effects.push(effect); // inside a transaction: remember it for later
  else void runEffects([effect]);    // outside: run now
}

```

`pendingEffects` is an `AsyncLocalStorage`: a Node feature that works like a variable scoped to "this chain of async calls". Any code deep inside the transaction can call `afterCommit(...)` without being passed anything, and the effect is attached to the *current* transaction. Only the attempt that commits runs its effects.

A rule this code follows everywhere: never call external services (send email, call the payment provider) *inside* a transaction. The transaction may be retried (the email would be sent twice) or rolled back (the email would describe something that didn't happen), and it holds locks while waiting on the network. Use `afterCommit` for best-effort effects and the **outbox** (chapter 13) for things that must happen.

CHAPTER 8

The journey of one request

Let's follow `POST /api/v1/events/:id/bookings` (hold seats) through the system. This table is the map of the whole backend:

Step	Where	What happens
1	Nginx	Decrypts HTTPS, applies an edge rate limit, adds <code>X-Request-Id</code> , picks the API copy with the fewest active connections.
2	<code>app.ts (genReqId)</code>	Adopts the request id (or makes one). Every log line of this request carries it.
3	<code>onRequest</code> hooks	Metrics: "one more in flight". Opens the request context so deep code can read the request id. Per-IP rate limit in Redis.
4	<code>auth/guard.ts</code>	Checks the access token, checks the session isn't revoked, sets <code>req.user</code> .
5	Zod schema	Validates <code>:id</code> and the body. Bad input never reaches the handler (400).
6	<code>bookings/routes.ts</code>	Per-user rate limit on holds; handles <code>Idempotency-Key</code> (safe retries).
7	<code>bookings/service.ts</code>	The business logic: checks, claim gate, transaction, lock seats, create the booking.
8	Kysely + pool	Borrows one connection for the transaction; fails fast with 503 if none is free in 5 s.
9	Serialization	The response schema decides which fields leave the server.
10	<code>onResponse</code> / <code>onSend</code>	Records the request duration; adds <code>x-request-id</code> and <code>x-served-by</code> headers.

A typical route definition shows steps 4, 5 and 9 in one place:

```
app.post(
  '/events/:id/bookings',
  {
    onRequest: requireAuth, // step 4: must be logged in
    schema: {
      tags: ['bookings'], // for the /docs page
      summary: 'Hold seats (creates a pending booking)',
      security: bearerAuth,
      params: IdParams, // step 5: :id must be a UUID
      headers: z.object({ 'idempotency-key': z.string().optional() }),
      body: z.object({
        seatIds: z.array(z.int().positive()).min(1).max(50)
          .refine((ids) => new Set(ids).size === ids.length, 'Seat ids must be unique'),
      }),
      response: { 201: BookingDto, ...errors, 429: ErrorResponse }, // step 9
    },
  },
  async (req, reply) => { // step 6-7: the handler
    const user = currentUser(req);
```

```

    await enforce(req, reply, [[HOLD_LIMIT, user.id]]);
    return withIdempotency(req, reply, user.id, async () => {
      const { bookingId } = await holdSeats(
        { userId: user.id, eventId: req.params.id, seatIds: req.body.seatIds },
        app.bookingOptions,
      );
      reply.header('location', `/api/v1/bookings/${bookingId}`);
      return { statusCode: 201, body: await getBookingFor(user, bookingId) };
    });
  },
);

```

Notice how **short** the handler is. All the thinking lives in `holdSeats` (the service). The route only connects HTTP to business logic.

The response schema is a security feature. Because Fastify serializes the response through the Zod schema, only declared fields are sent. Even if a handler accidentally returned a whole database row including `password_hash`, it would not leave the server. The code also converts rows to explicit "DTOs" (data transfer objects) like `toUserDto` for the same reason.

Validation and errors

Validation with Zod

Any data coming from outside is untrusted. Zod checks it at the door. From `auth/routes.ts`:

```
body: z.object({
  email: z.email().max(254),
  password: Password, // z.string().min(8).max(128)
  name: z.string().trim().min(1).max(100),
  role: z.enum(['attendee', 'organizer']).default('attendee'), // no "admin" from the public
}),
```

If someone sends `{ "email": "nope", "password": "123" }`, the handler never runs; the client gets:

```
{ "error": { "code": "VALIDATION_ERROR", "message": "Invalid request body",
"details": [ { "path": "body.email", "message": "Invalid email address" },
              { "path": "body.password", "message": "Use at least 8 characters" } ] } }
```

Complex rules use `.refine` / `.superRefine`, like "section names must be unique" or "endsWith must be after startsAt" (see `events/schemas.ts` and `venues/routes.ts`).

One error shape (`src/lib/errors.ts`)

Every error response in the app has the same shape: `{ "error": { "code", "message", "details?" } }`. Clients branch on `code` (stable), and `message` is for humans.

The code defines an error class and small helper functions:

```
export class AppError extends Error {
  constructor(
    readonly statusCode: number,
    readonly code: string,
    message: string,
    readonly details?: unknown,
  ) {
    super(message);
  }
}

export const notFound = (resource: string) => new AppError(404, 'NOT_FOUND', `${resource} not found`);
export const conflict = (code: string, message: string, details?: unknown) =>
  new AppError(409, code, message, details);
```

So anywhere in the code you can write `throw notFound('Event')` or `throw conflict('SEATS_UNAVAILABLE', 'Some seats are no longer available', { seatIds })`, and the **central error handler** converts it into a response. Handlers never build error responses themselves.

The error handler (`errorHandler`) is a chain of "what kind of error is this?":

1. **Zod validation error** → 400 `VALIDATION_ERROR` with per-field details.
2. **AppError** → its own status and code (401s also get a `WWW-Authenticate` header).
3. **Postgres constraint error**, mapped by the Postgres error code:

```
const PG_ERRORS = {  
  '23505': { status: 409, code: 'DUPLICATE', ... },           // unique violation  
  '23503': { status: 409, code: 'REFERENCE_CONFLICT', ... }, // foreign key  
  '23P01': { status: 409, code: 'EXCLUSION_CONFLICT', ... }, // e.g. overlapping events  
  '23514': { status: 422, code: 'CONSTRAINT_VIOLATION', ... }, // CHECK failed  
};
```

This is the "database as the last safety net" idea: even if a handler forgot a check, the constraint fires and the client gets a clean 4xx instead of a 500.

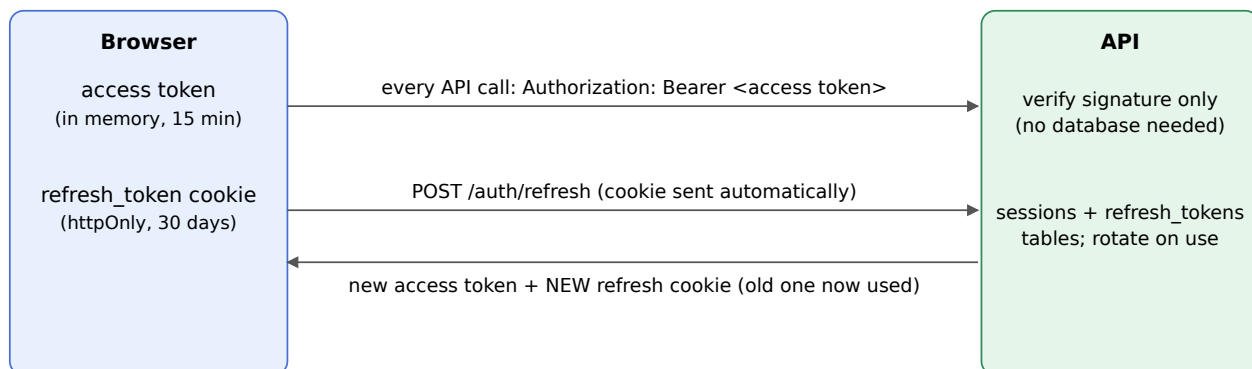
1. **Deadlock / serialization failure** that survived retries → 503 `TRANSIENT_CONFLICT` with `Retry-After: 1`.
2. **Pool timeout or statement timeout** → 503 `SERVICE_BUSY` with `Retry-After: 2`.
Overload is not a bug; the client should back off.
3. Fastify's own client errors (bad JSON, body too large) → a stable code.
4. Anything else → logged as a bug, and the client sees a generic 500 `INTERNAL_ERROR` (never the stack trace, which could leak internals).

Lesson: centralise error handling. Throw meaningful errors from anywhere; convert them to HTTP in exactly one place. Your handlers stay clean, and every error looks the same to clients.

Authentication: who are you?

Authentication = proving who you are (logging in). **Authorization** = deciding what you may do (roles, ownership). The code for both is in `src/modules/auth/`. This project writes auth by hand (no auth service) specifically to learn it.

The big picture



Two tokens: a short-lived one for every request, a long-lived one only for getting new short ones.

Step 1: storing passwords (`passwords.ts`)

Never store a password. Store a *hash*: a one-way scramble. At login, you hash what the user typed and compare. This project uses **Argon2id**, which is deliberately slow and memory-hungry (19 MiB per guess), so an attacker who steals the database can't try billions of passwords per second on GPUs.

```

const PARAMS = {
  algorithm: 2 as Algorithm.Argon2id,
  memoryCost: config.ARGON2_MEMORY_KIB, // 19,456 KiB
  timeCost: config.ARGON2_TIME_COST, // 2 passes
  parallelism: 1,
};

/** Returns a self-describing string: $argon2id$v=19$m=19456,t=2,p=1$<salt>$<hash> */
export const hashPassword = (password: string) => hash(password, PARAMS);

export async function verifyPassword(phc: string, password: string): Promise<boolean> {
  try {
    return await verify(phc, password);
  } catch {
    return false; // malformed hash: treat as a failed login, never as a crash
  }
}

```

Two clever details:

- `needsRehash(phc)` checks whether a stored hash used older settings. On a successful login (the only moment you know the real password) the app re-hashes with the new settings. So you can make hashing stronger later without forcing everyone to reset.
- `burnVerifyTime(password)` : when the email doesn't exist, the app still spends the same time as a real password check. Otherwise an attacker could measure response times to learn which emails have accounts.

Step 2: the access token (JWT, `tokens.ts`)

After login, the server gives the client an **access token**, a **JWT** (JSON Web Token). A JWT is three base64 parts: `header.payload.signature`. The payload contains *claims*:

```
{ "sub": "<user id>", "sid": "<session id>", "role": "attendee", "exp": 1767225600, "iss": "ticket-mng", "aud": "ticket-mng-api" }
```

The signature is an HMAC made with a secret only the server knows (`JWT_ACCESS_SECRET`). So the server can check a token **without looking anything up in the database**: if the signature matches, the server itself issued it and nobody changed it.

```
export function signAccessToken(claims: AccessClaims): Promise<string> {
  return new SignJWT({ sid: claims.sid, role: claims.role })
    .setProtectedHeader({ alg: 'HS256', typ: 'JWT' })
    .setSubject(claims.sub)
    .setIssuer(ISSUER)
    .setAudience(AUDIENCE)
    .setIssuedAt()
    .setExpirationTime(`${config.ACCESS_TOKEN_TTL_SECONDS}s`) // 15 minutes
    .sign(keys[0]!);
}

export async function verifyAccessToken(token: string): Promise<AccessClaims> {
  for (const key of keys) {
    try {
      const { payload } = await jwtVerify(token, key, {
        algorithms: ['HS256'], // pinned: the token can't choose how it's verified
        issuer: ISSUER,
        audience: AUDIENCE,
      });
      // ... validate claims with Zod and return them
    } catch (err) {
      if (err instanceof joseErrors.JWSSignatureVerificationFailed) continue; // try previous
      if (err instanceof joseErrors.JWTExpired) throw unauthorized('TOKEN_EXPIRED', '...');
      break;
    }
  }
  throw unauthorized('INVALID_TOKEN', 'Access token is invalid');
}
```

- `algorithms: ['HS256']` is important. Some JWT libraries used to trust the algorithm written *in the token*, and attackers sent tokens with `"alg": "none"` (no signature). Pinning the algorithm closes that hole.
- `keys` holds the current secret and optionally the previous one, so you can change the secret without logging everyone out (key rotation).

The problem with JWTs: once issued, a JWT is valid until it expires; you can't "un-issue" it. That's why it lives only **15 minutes**, and why there is a second token.

Step 3: sessions and the refresh token (sessions.ts)

Each login creates a row in `sessions` (which device, when, from which IP) and a **refresh token**: 32 random bytes. The token is given to the browser in a cookie, and only its **SHA-256 hash** is stored in `refresh_tokens` (so a stolen database can't be used to log in).

```
export async function startSession(conn, user, meta): Promise<IssuedTokens> {
  const expiresAt = new Date(Date.now() + config.SESSION_MAX_DAYS * DAY_MS);
  const { id } = await conn
    .insertInto('sessions')
    .values({ userId: user.id, expiresAt, userAgent: meta.userAgent?.slice(0, 300) ?? null, ip: meta.ip ?? null })
    .returning('id')
    .executeTakeFirstOrThrow();
  return issueTokens(user, id, await issueRefreshToken(conn, id, expiresAt));
}
```

When the access token expires, the browser calls `POST /auth/refresh`. The cookie is sent automatically. `rotateRefreshToken`:

1. finds the token row by its hash **and locks it** (`FOR UPDATE`) so two simultaneous refreshes can't both use it;
2. if the session was revoked or expired → 401;
3. **if the token was already used** (more than 10 seconds ago) → someone copied it! The **whole session is revoked**. This is *theft detection*: the thief and the real user end up presenting the same token, and whoever comes second trips the alarm;
4. otherwise marks it used, issues a **new** refresh token (rotation) and a new access token. The role is re-read from the database, so role changes take effect within 15 minutes.

```
if (row.usedAt && now - row.usedAt.getTime() > REUSE_LEEWAY_MS) {
  await trx.updateTable('sessions')
    .set({ revokedAt: new Date(), revokeReason: 'refresh_token_reuse' })
    .where('id', '=', row.sessionId)
    .execute();
  return { kind: 'reused', sessionId: row.sessionId } as const;
}
```

Notice it *returns* an outcome instead of throwing inside the transaction. Throwing would roll back the transaction, undoing the revocation it just made.

Step 4: the cookie (auth/routes.ts)

```
reply.setCookie(REFRESH_COOKIE, tokens.refreshToken, {
  httpOnly: true, // JavaScript can't read it → an XSS bug can't steal it
  secure: config.COOKIE_SECURE, // HTTPS only (in production)
  sameSite: 'strict', // not sent on requests from other websites → blocks CSRF
  path: '/api/v1/auth', // only sent to auth endpoints, not with every API call
  expires: tokens.refreshExpiresAt,
});
```

The access token, on the other hand, is returned in the JSON body and kept in a JavaScript variable (memory), not in `localStorage`.

Step 5: immediate logout (the denylist)

If access tokens need no database lookup, how does logout take effect *now* instead of in 15 minutes? The session id (`sid`) is inside the token. On logout, the app marks the session revoked in Postgres **and** writes `auth:revoked-session:<id>` into Redis for 15 minutes. The guard checks that key on every request.

```
export async function isSessionRevoked(sessionId: string): Promise<boolean> {
  try {
    return (await redis.exists(denyKey(sessionId))) === 1;
  } catch (err) {
    logger.error({ err }, 'session denylist unavailable');
    return false; // fail open: a Redis outage shouldn't lock everyone out
  }
}
```

"Fail open" is a conscious trade-off: if Redis is down, just-revoked tokens keep working up to 15 minutes, rather than *nobody* being able to use the site.

Step 6: the guard (`guard.ts`)

Routes declare who can call them with `onRequest: requireAuth` or `onRequest: requireRole('organizer', 'admin')`:

```
async function authenticate(req: FastifyRequest): Promise<AuthUser> {
  const header = req.headers.authorization;
  if (!header) throw unauthorized();
  const [scheme, token] = header.split(' ');
  if (scheme?.toLowerCase() !== 'bearer' || !token) throw unauthorized('INVALID_TOKEN', '...');

  const claims = await verifyAccessToken(token);
  if (await isSessionRevoked(claims.sid)) throw unauthorized('SESSION_REVOKED', '...');
  req.log = req.log.child({ userId: claims.sub }); // later log lines name the user
  return { id: claims.sub, role: claims.role, sessionId: claims.sid };
}

export async function requireAuth(req: FastifyRequest) { req.user = await authenticate(req); }

export function requireRole(...roles: UserRole[]) {
  return async (req: FastifyRequest) => {
    req.user = await authenticate(req);
    if (!roles.includes(req.user.role)) throw forbidden();
  };
}

export function canManage(user: AuthUser | null, ownerId: string): boolean {
  return !!user && (user.role === 'admin' || user.id === ownerId);
}
```

- These run as `onRequest` hooks, **before the body is parsed**, so an unauthenticated flood is rejected cheaply.

- `optionalAuth` is for public routes that show more to logged-in users: no token is fine, but a *bad* token is an error (never silently treat a broken token as anonymous).
- **Roles** answer "what kind of user?". **Ownership** (`canManage`) answers "is this *your* event?". Organizers manage their own events; admins manage everything.
- **404 instead of 403 for hidden things.** In `events/access.ts` , a draft event you don't own returns 404 "not found", not 403 "forbidden", so outsiders can't even learn it exists.

Other auth features worth reading

- **Rate limits on login** (`LIMITS` in `auth/routes.ts`): per IP, and per IP+email, so password guessing is slowed without letting an attacker lock a victim out.
- **Same error for unknown email and wrong password** (`INVALID_CREDENTIALS`), so the API doesn't reveal which emails are registered.
- **Password reset:** `POST /auth/password-reset/request` always answers 202 and does *nothing visible*; it queues a job, and the **worker** looks up the account and emails a single-use, 30-minute token. Doing the lookup in the worker means response time can't reveal whether the account exists. The link carries the token in the URL *fragment* (`#token=...`), which browsers never send to servers.
- **Password change / role change / reset** revoke sessions so the change takes effect everywhere.

Venues and events

Creating a venue and generating seats

An organizer sends a compact description:

```
{ "name": "City Hall", "address": "1 Main St", "city": "Springfield", "country": "US",
  "sections": [ { "name": "Floor", "rows": 20, "seatsPerRow": 30 },
                 { "name": "Balcony", "rows": 8, "seatsPerRow": 40 } ] }
```

`venues/layout.ts` turns that into individual seats with coordinates for drawing. Row labels go A, B, ... Z, AA, AB like spreadsheet columns:

```
export function rowLabel(index: number): string {
  let label = '';
  let n = index + 1;
  while (n > 0) {
    const rem = (n - 1) % 26;
    label = String.fromCharCode(65 + rem) + label; // 65 is 'A'
    n = Math.floor((n - 1) / 26);
  }
  return label;
}

export function generateSeats(sections: SectionSpec[]): GeneratedSeat[] {
  const maxWidth = Math.max(...sections.map((s) => s.seatsPerRow));
  const seats: GeneratedSeat[] = [];
  let y = 0;
  for (const section of sections) {
    const xOffset = Math.floor((maxWidth - section.seatsPerRow) / 2); // centre narrower sections
    for (let r = 0; r < section.rows; r++) {
      const label = rowLabel(r);
      for (let n = 1; n <= section.seatsPerRow; n++) {
        seats.push({ section: section.name, rowLabel: label, seatNumber: n, x: xOffset + n - 1,
y });
      }
      y++;
    }
    y += SECTION_GAP; // empty rows between sections
  }
  return seats;
}
```

This is a **pure function** (input → output, no database, no network), which is why it has simple unit tests in `test/unit/layout.test.ts`. The route then saves the venue, its sections and all seats in one transaction, using the `unnest` bulk insert shown in chapter 7.

Creating an event (`events/routes.ts`)

`POST /events` receives the venue id, times, and a price per section. The handler:

1. checks every venue section is priced exactly once (otherwise 422 `INVALID_PRICING`);

2. in a transaction: **locks the venue row** (`lockVenueSchedule`), inserts the event as a **draft**, and copies all venue seats into `event_seats` with their section's price, in one SQL statement:

```
await sql`
  INSERT INTO event_seats (event_id, venue_seat_id, price_cents)
  SELECT ${id}::uuid, vs.id, p.price_cents
  FROM venue_seats vs
  JOIN venue_sections sec ON sec.id = vs.section_id
  JOIN unnest(${pricing.map((p) => p.section)}::text[], ${pricing.map((p) =>
p.priceCents)}::int[])
  AS p(section_name, price_cents) ON p.section_name = sec.name
  WHERE sec.venue_id = ${venueId}::uuid
`.execute(trx);
```

1. if the overlap constraint fires (error `23P01`), it becomes a friendly `409` `VENUE_TIME_CONFLICT` .

Why lock the venue row first? The comment in the code explains a real bug they hit: two simultaneous inserts for overlapping times could each wait for the other inside the exclusion-constraint check (a deadlock that Postgres only breaks after 1 second). Locking the venue row makes them queue up instead, and the second one fails fast and cleanly. This is a good example of a problem you only find by testing concurrency ("20 parallel creates for one slot, exactly one wins" in `test/api/events.test.ts`).

Event status: a small state machine

```
export const STATUS_TRANSITIONS: Record<EventStatus, readonly EventStatus[]> = {
  draft: ['published', 'cancelled'],
  published: ['cancelled'],
  cancelled: [],
};
```

`PATCH /events/:id` locks the event row (`FOR UPDATE`) so two concurrent edits can't both read "draft" and apply conflicting changes, then checks the transition is allowed. Cancelling an event also cancels all pending holds and queues a `refund-event` job that refunds every paid booking.

Pattern: explicit state machines. Whenever something has a status (orders, bookings, payments, tickets), write down which transitions are allowed, as data, and check them. It prevents impossible states like "cancelled, then published again".

Listing and searching events: cursor pagination

`GET /events?q=rock&city=Berlin&limit=20` searches with Postgres full-text search (`e.search @@ websearch_to_tsquery('english', q)`) and returns pages.

There are two ways to paginate:

- **Offset** ("skip 100, take 20"): simple, used for venues. But `OFFSET 100000` makes the database read and discard 100,000 rows, and if rows are added between pages, items shift and repeat.
- **Keyset / cursor** ("give me 20 events that come *after* the last one I saw"): used for events. It uses the index to jump straight there, so page 5,000 costs the same as page 1.

```
if (cursor) {
  const c = decodeCursor(cursor);
  query = query.where(sql<boolean>`(e.starts_at, e.id) > (${c.at}::timestampz,
${c.id}::uuid)`);
}
const rows = await query.orderBy('e.startsAt').orderBy('e.id').limit(limit + 1).execute();

const hasMore = rows.length > limit;           // fetched one extra: is there a next page?
const page = hasMore ? rows.slice(0, limit) : rows;
const last = page.at(-1);
return {
  data: page.map(toEventDto),
  page: { limit, nextCursor: hasMore && last ? encodeCursor({ at: last.startsAt, id: last.id })
: null },
};
```

- Sorting by `(starts_at, id)` makes the order total (the id breaks ties between events at the same time), so nothing is skipped.
- Asking for `limit + 1` rows is a neat trick to know whether another page exists without a second `COUNT` query.
- The cursor (in `lib/pagination.ts`) is just `{t, id}` as JSON, base64url-encoded, so clients treat it as an opaque string.

Reading an event and its seat map

`GET /events/:id` returns the event plus live availability and a price range. `GET /events/:id/seats` returns every seat with position, price, status and version. These are the most-read endpoints during a sale, so they're cached (chapter 19). One important detail in the seat map query:

```
sql<SeatStatus>`CASE WHEN ${acquirableSql} THEN 'available'::seat_status ELSE es.status
END`.as('status')
```

A seat whose hold has **lapsed** is shown as available right away, even before any background job has cleaned it up. That's "lazy expiry", explained in the next chapter.

Bookings: the hardest part

Open `src/modules/bookings/service.ts`. The big comment at the top is worth reading slowly. This chapter explains it from zero.

The problem: a race condition

Holding a seat sounds simple:

```
1. read the seat      → status = 'available'
2. if available, write → status = 'held', booking_id = my booking
```

Now imagine two buyers, A and B, clicking the same seat at the same millisecond. Their steps can interleave:

```
A: read seat → available
B: read seat → available      (A hasn't written yet!)
A: write     → held by A      → A gets "201 Created"
B: write     → held by B      → B gets "201 Created" ← both think they own it!
```

That's a **race condition**, and it's exactly how double-booking happens. The project proves it: `npm run race` sends 200 simultaneous requests for one seat. With the deliberately broken `naive` strategy, **11 people** got the seat. With the correct strategies, exactly **1**.

Four strategies (all in the code, for comparison)

Strategy	Idea	Result in the race test
<code>naive</code>	read, check, write, no locking	11 owners (broken on purpose, refused in production)
<code>optimistic</code>	no locks; write only if the version is still what I read	1 owner
<code>serializable</code>	strictest isolation level; Postgres aborts conflicting transactions	1 owner
<code>pessimistic</code> (default)	lock the rows while reading: <code>SELECT ... FOR UPDATE SKIP LOCKED</code>	1 owner
<code>pessimistic</code> + claim gate	first a fast Redis check, then the above	1 owner, and only 30 of 200 reached Postgres

Optimistic locking uses the `version` column. The write says "update this seat only if its version is still 7". If someone changed it in between, the version is 8, zero rows match, and we report a conflict:

```
UPDATE event_seats es
SET status = 'held', booking_id = $1, version = es.version + 1
```

```
FROM unnest($2::bigint[], $3::int[]) AS seen(id, version)
WHERE es.id = seen.id AND es.version = seen.version
RETURNING es.id, es.event_id, es.status, es.version
```

(`$1` is the new booking's id, `$2` the seat ids and `$3` the versions that were read earlier; in the code they are passed with Kysely's `sql` tag.)

Pessimistic locking takes a row lock while reading. `FOR UPDATE` means "these rows are mine until I commit". Normally, other transactions would *wait* for the lock. `SKIP LOCKED` tells them instead to *skip* locked rows. So a loser sees fewer seats than it asked for and fails instantly with 409, instead of waiting in line while holding a database connection:

```
async pessimistic(req, event, opts) {
  return withTransaction(async (trx) => {
    const rows = await seatQuery(trx, req.seatIds)
      .where('es.eventId', '=', req.eventId)
      .forUpdate('es')           // lock these seat rows
      .skipLocked()              // ...but skip ones someone else has locked right now
      .execute();
    if (rows.length < req.seatIds.length) {
      await assertSeatsExist(trx, req.eventId, req.seatIds, rows); // 422 if ids are bogus
      throw conflict('SEATS_UNAVAILABLE', 'Some seats are being booked by someone else right now', {...});
    }
    assertAcquirable(rows);      // 409 if a seat is already held or booked
    return createHold(trx, req, event, rows, opts, 'plain');
  });
},
```

The claim gate (`claims.ts`): a Redis shield in front

Even with `SKIP LOCKED`, 200 requests all open a database transaction. During a big sale, database connections are the scarcest resource. So before touching Postgres, each request tries to **claim** its seats in Redis with `SET key value NX PX 5000` ("set only if it doesn't exist, expire after 5 seconds"). This runs as a small **Lua script** inside Redis, so claiming several seats is all-or-nothing and atomic:

```
for i, key in ipairs(KEYS) do
  if not redis.call('SET', key, ARGV[1], 'NX', 'PX', ARGV[2]) then
    -- someone else holds this seat: undo the claims I made so far
    for j = 1, i - 1 do
      if redis.call('GET', KEYS[j]) == ARGV[1] then redis.call('DEL', KEYS[j]) end
    end
    return i           -- which seat conflicted
  end
end
return 0              -- all claimed
```

199 of the 200 get turned away in about 0.1 ms without touching the database.

The claim gate is a load shield, NOT the lock. A Redis key with a timeout can expire while its owner is still working, or be lost if Redis restarts. So the database transaction still decides who wins. If Redis is down, the gate is simply skipped (`catch → return { ok: true }`): more database load, never a wrong answer. Each claim stores a random token and is only deleted by the same token, so a slow request can never release someone else's claim.

holdSeats : the whole flow

```
export async function holdSeats(req: HoldRequest, opts: BookingOptions) {
  const seatIds = [...new Set(req.seatIds)].sort((a, b) => a - b); // dedupe + sort
  const request = { ...req, seatIds };
  const event = await loadSaleableEvent(req.eventId, seatIds.length); // published? on sale?
  await checkUserLimits(request, event); // existing hold? ticket limit?

  const claim = opts.claimGate ? await claimSeats(req.eventId, seatIds) : null;
  if (claim && !claim.ok) {
    holdAttempts.inc({ strategy: opts.strategy, outcome: 'gate_rejected' });
    throw conflict('SEATS_UNAVAILABLE', '...', { seatIds: [claim.conflictingSeatId] });
  }

  try {
    const result = await strategies[opts.strategy](request, event, opts);
    holdAttempts.inc({ strategy: opts.strategy, outcome: 'success' });
    return result;
  } catch (err) {
    // ... count the failure; turn the "one pending hold per user" index violation into HOLD_EXISTS
    throw err;
  } finally {
    await claim?.release(); // always release the Redis claims
  }
}
```

The order is deliberate: **cheap checks first** (no locks: is the event on sale? does this user already have a hold?), then the Redis gate, and only then the expensive locked transaction.

`createHold` (called inside the transaction) does the actual writes:

1. `INSERT INTO bookings` with status `pending` and `expires_at = now() + 10 minutes` (using the **database's** clock, not the server's);
2. `enqueue(...)` an `expire-booking` job for that time (through the outbox, in the same transaction);
3. `UPDATE event_seats SET status='held', booking_id=..., version=version+1` and pass the changed rows to `changed(...)`, which publishes them to live seat maps **after commit**;
4. `INSERT INTO booking_items` (which seats, at which price);
5. if any of the seats were taken over from a *lapsed* hold, mark that old booking `expired`.

Lazy expiry: holds expire without waiting for a job

A hold lasts 10 minutes. The obvious design: a timer job flips it to expired. But what if the worker is down or late? The seat would stay blocked. This project does something smarter. A seat counts as **free** if it's available **or** held by a booking that is no longer an active hold:

```
export const acquirableSql = sql<boolean>`(es.status = 'available' OR
(es.status = 'held' AND (b.status <> 'pending' OR b.expires_at <= now())))`;
```

This exact expression is used everywhere availability matters: the seat map, the availability count, placing a hold, confirming a payment. So the moment `expires_at` passes (by the database clock, which all servers share), the seat is free. The `expire-booking` job and a 30-second sweep only *tidy up* the rows and notify live viewers. **Correctness never waits for a background job.**

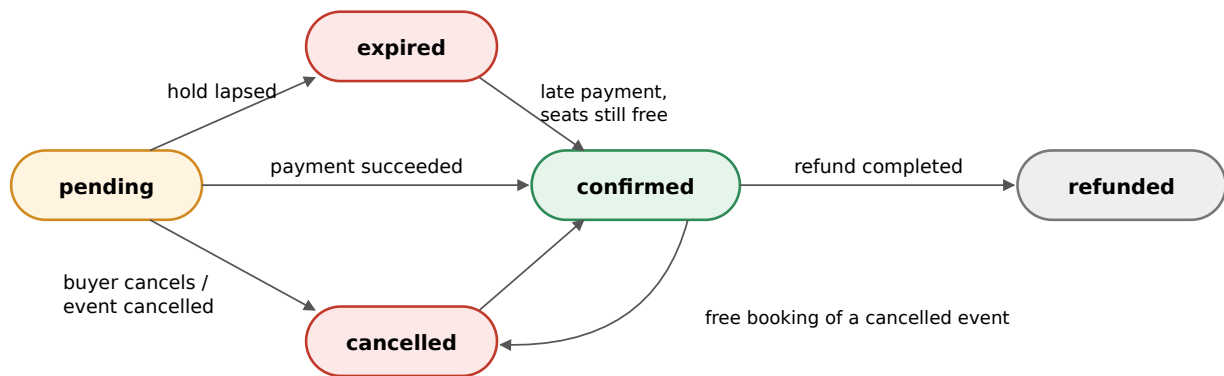
Deadlocks and the global lock order

A **deadlock**: transaction 1 locks seat A and wants booking X; transaction 2 locks booking X and wants seat A. Each waits for the other forever (Postgres detects it and kills one).

The fix is simple and powerful: **always lock things in the same order**. In this codebase, every operation that touches bookings (hold, pay, cancel, expire, refund, cancel event) locks **seat rows first (in ascending id order), then the booking, then the payment**. With one global order, a cycle is impossible. That's what `lockBooking` does:

```
export async function lockBooking(trx: Transaction<DB>, bookingId: string) {
  const items = await trx.selectFrom('bookingItems').select('eventSeatId')
    .where('bookingId', '=', bookingId).orderBy('eventSeatId').execute();
  const seatIds = items.map(i => i.eventSeatId);
  const seats = seatIds.length ? await seatQuery(trx, seatIds).forUpdate('es').execute() : [];
  // 1. seats
  const booking = await trx.selectFrom('bookings').selectAll()
    .select(sql<boolean>`expires_at <= now()`).as('lapsed')
    .where('id', '=', bookingId)
    .forUpdate()
  // 2. booking
    .executeTakeFirst();
  return { seats, booking, seatIds };
}
```

The life of a booking



Booking states. Holds become confirmed when paid; lapsed or cancelled holds can still be confirmed by a late payment if the seats are free.

The functions that move between states, all in `bookings/service.ts` :

- `expireBooking(id)` : pending + lapsed → expired , release seats. Safe to call many times ("idempotent").
- `cancelPendingBooking(id)` : the buyer walks away → cancelled , release seats.
- `confirmBookingInTx(trx, id, { lateAllowed })` : pending → confirmed , seats held → booked , **issue tickets**, and queue the confirmation email, all in the caller's transaction. With `lateAllowed` (used by payments), if the hold lapsed but the seats are still free, it takes them back instead of refunding: money has already moved, so honour the purchase if possible. It returns an *outcome* (confirmed , seats_lost , hold_expired , event_unavailable , ...) so the payment code can decide to refund.
- `endConfirmedBookingInTx(trx, id, 'refunded' | 'cancelled')` : void tickets and release seats.

Big lessons from this chapter. (1) Any "check then act" on shared data is a race unless something locks or verifies it. (2) Let the database decide; use caches only to reduce load. (3) Lock in one global order. (4) Don't depend on timers for correctness; compute state from timestamps. (5) Prove it with a test that fires many concurrent requests.

Background jobs and the outbox

The "dual write" problem

When a booking is confirmed, two things must happen: the database saves the booking, and an email job is added to the Redis queue. Those are **two different systems**. Whatever order you pick, a crash in the middle is a bug:

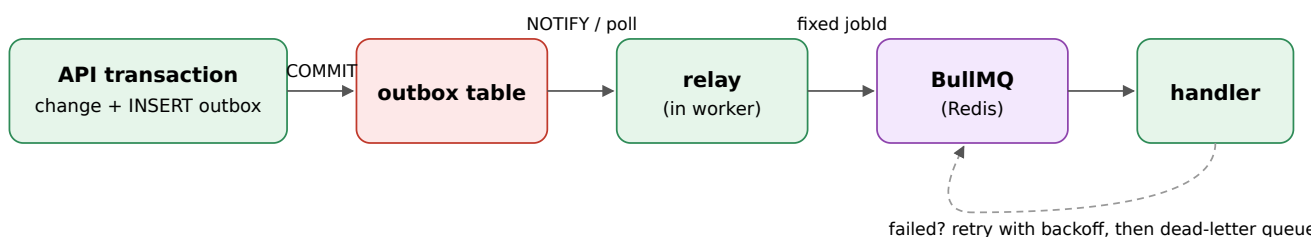
- save to DB, *crash*, never enqueue → the customer paid but never gets tickets;
- enqueue first, then the DB transaction *rolls back* → an email for a booking that doesn't exist.

The solution: the transactional outbox (`src/jobs/outbox.ts`)

Code **never** talks to the queue directly. Instead it inserts a row into an `outbox` **table**, in the **same transaction** as the business change:

```
export async function enqueue(conn, queue, name, data, opts = {}): Promise<void> {
  await conn
    .insertInto('outbox')
    .values({
      queue,
      jobName: name,
      payload: JSON.stringify(data),
      jobId: opts.jobId ?? null,           // dedupe key
      runAt: opts.runAt ?? sql<Date>`now()`, // for delayed jobs
      requestId: currentRequestId() ?? null, // links the job's logs to the request
    })
    .execute();
}
```

Both commit or neither does. Then a **relay** in the worker process moves committed rows into BullMQ:



```
export async function publishOutboxBatch(): Promise<number> {
  return withTransaction(async (trx) => {
    const rows = await trx.selectFrom('outbox').selectAll()
      .where('publishedAt', 'is', null)
      .orderBy('id').limit(BATCH)
      .forUpdate().skipLocked() // several relays can run without taking the same rows
      .execute();
    if (!rows.length) return 0;
  });
}
```

```
// group by queue and add to BullMQ with a FIXED job id
for (const [queue, items] of byQueue) {
  await getQueue(queue).addBulk(items.map((r) => ({
    name: r.jobName,
    data: { ...r.payload, _meta: { requestId: r.requestId, outboxId: r.id } },
    opts: { jobId: r.jobId ?? `outbox-${r.id}`, delay: Math.max(0, r.runAt.getTime() - now)
  })),
  }));
  await trx.updateTable('outbox').set({ publishedAt: new Date() }).where('id', 'in',
rows.map((r) => r.id)).execute();
  return rows.length;
}, { retries: 0 });
}
```

Why every piece is there:

- `FOR UPDATE SKIP LOCKED` (again!): if you run several workers, each takes different rows.
- **Fixed job id**: if the relay crashes *after* adding to Redis but *before* marking rows published, it will publish them again. BullMQ ignores a job whose id already exists, so the duplicate is harmless.
- **How the relay wakes up**: a database trigger runs `pg_notify('outbox')` when a row is inserted; the relay `LISTEN`s and reacts within milliseconds. It also polls every 2 seconds in case a notification is missed. (Migration 0006 is a lesson from load testing: notifying on every insert slowed commits, so now only jobs due immediately notify; delayed ones are found by polling.)

This gives **at-least-once delivery**: a job may run twice, but never zero times. So every handler must be **idempotent** (safe to run twice).

Queues and job types (`src/jobs/queues.ts`)

All job types are declared in one TypeScript interface, so a typo in a job name or a wrong payload is a compile error:

```
export interface Jobs {
  email: {
    'password-reset': { email: string };
    'booking-confirmed': { bookingId: string };
    'event-reminder': { bookingId: string };
    'refund-processed': { refundId: string };
  };
  payments: {
    'process-webhook': { provider: string; eventId: string };
    refund: { refundId: string };
    'refund-event': { eventId: string };
  };
  bookings: { 'expire-booking': { bookingId: string }; 'sweep-expired-holds': Record<string, never> };
  media: { 'process-poster': { eventId: string; key: string } };
  maintenance: { 'send-event-reminders': Record<string, never>; cleanup: Record<string, never> };
}
```

Payloads carry **ids only**, never copies of data. By the time a job runs (maybe 10 minutes later), the data may have changed, so the handler re-reads the current state.

Retry policy: 5 attempts with exponential backoff and jitter (about 2 s, 4 s, 8 s, 16 s, each $\pm 50\%$), so a burst of jobs that failed together (the email server blipped) doesn't retry in lockstep and knock it over again.

`src/jobs/handlers/index.ts` maps every job name to its handler function, and its type makes TypeScript complain if you declare a job but forget its handler.

The runner and the dead-letter queue (`runner.ts`)

`createWorker(queue, handlers, concurrency)` builds a BullMQ worker that looks up the handler by job name, gives it a logger tagged with the queue, job id, attempt number and **the original request id**, measures duration for metrics, and, when a job has failed for the last time, copies it into a **dead-letter queue**:

```
worker.on('failed', (job, err) => {
  if (!job || !isFinalFailure(job, err)) return;
  sendToDeadLetter(job, err).catch(...);
});
```

Nothing consumes the dead-letter queue. It waits for a human: `GET /api/v1/admin/dead-letters` lists failed jobs with their error, and admins can retry or discard them. Failures don't silently disappear.

Idempotent handlers: sending an email once (`handlers/email.ts`)

```
export async function sendOnce(kind, refId, userId, send): Promise<'sent' | 'already_sent'> {
  return db.transaction().execute(async (trx) => {
    await trx.insertInto('notifications').values({ kind, refId, userId })
      .onConflict((oc) => oc.columns(['kind', 'refId']).doNothing())
      .execute();
    const row = await trx.selectFrom('notifications').select('status')
      .where('kind', '=', kind).where('refId', '=', refId)
      .forUpdate() // a duplicate job waits here
      .executeTakeFirstOrThrow();
    if (row.status === 'sent') return 'already_sent';
    await send(); // talk to the SMTP server
    await trx.updateTable('notifications').set({ status: 'sent', sentAt: sql`now()` })
      .where('kind', '=', kind).where('refId', '=', refId).execute();
    return 'sent';
  });
}
```

Each email has a natural key (for example `booking-confirmed` + `booking id`). The row is locked during the send, so a concurrent duplicate waits and then sees `sent`. The only remaining gap is a crash *after* the email server accepted the message but *before* the commit; that's the unavoidable edge of at-least-once delivery, and it's documented honestly.

(This is the one intentional exception to "no network calls inside a transaction": here the lock is exactly what guarantees once-only sending.)

Scheduled jobs (`schedules.ts`)

```
await getQueue('bookings').upsertJobScheduler('sweep-expired-holds', { every: 30_000 }, {...});
await getQueue('maintenance').upsertJobScheduler('send-event-reminders', { every: 15 * 60_000 }, {...});
await getQueue('maintenance').upsertJobScheduler('cleanup', { pattern: '0 3 * * *', tz: 'UTC' }, {...});
```

`upsert` means "create or update": every worker can call it at startup and Redis still holds exactly one schedule, so jobs don't run N times with N workers. `'0 3 * * *'` is cron syntax: minute 0, hour 3, every day.

The handlers:

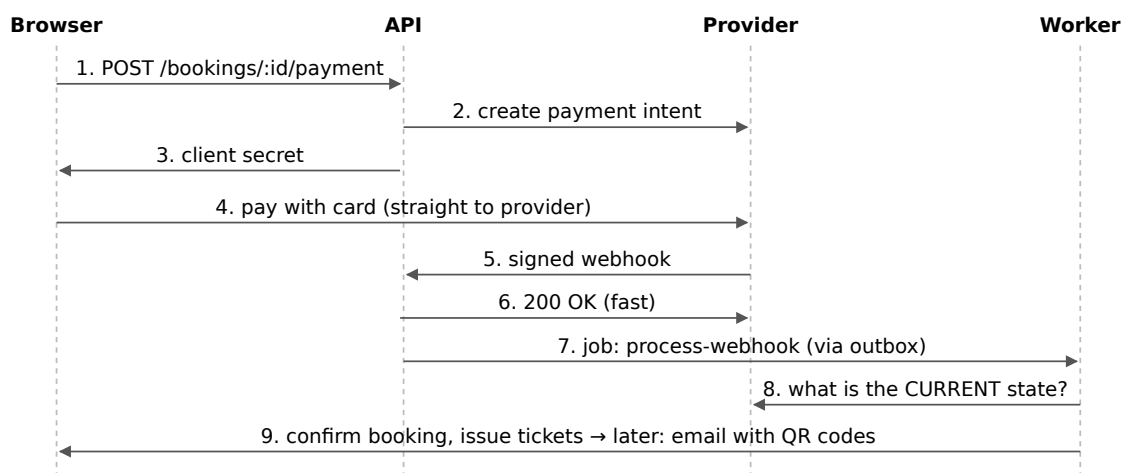
- `sweep-expired-holds` (every 30 s): finds pending bookings past their expiry and seats still "held" by non-pending bookings, and releases them. A safety net for lost expiry jobs.
- `send-event-reminders` (every 15 min): emails attendees before their event.
- `cleanup` (daily): deletes old outbox rows, expired tokens, stale idempotency keys and similar.

Payments and webhooks

Money is where bugs are most expensive. Open `src/modules/payments/service.ts` and read its top comment alongside this chapter.

How card payments work (the general idea)

Your server **never sees card numbers**. The flow with any modern provider (Stripe, and the fake gateway here, which copies Stripe's shape):



1. The browser asks our API to start paying for a pending booking.
2. The API creates a **payment intent** at the provider ("I want to charge 45.00 USD").
3. The API returns the intent's **client secret** to the browser.
4. The browser sends the card details **directly to the provider** with that secret.
5. The provider tells our server what happened by calling our URL: a **webhook**.
6. We answer 200 immediately.
7. A background job processes it.
8. The job asks the provider for the payment's **current** state.
9. If it succeeded: confirm the booking, issue tickets, queue the email.

One interface, two providers (`providers/`)

```

export interface PaymentProvider {
  readonly name: 'fake' | 'stripe';
  createPayment(input: CreatePaymentInput, idempotencyKey: string): Promise<ProviderPayment & {
    clientSecret: string }>;
  retrievePayment(providerPaymentId: string): Promise<ProviderPayment>;
  refundPayment(providerPaymentId: string, amountCents: number, idempotencyKey: string):
    Promise<{...}>;
  parseWebhook(rawBody: Buffer, headers: Record<string, string | string[] | undefined>):
    ProviderEvent;
}
  
```

`providers/stripe.ts` and `providers/fake.ts` both implement this; `providers/index.ts` picks one based on `PAYMENT_PROVIDER`. The rest of the code never knows which one is used. This is the **adapter pattern**: wrap an external service behind your own small interface, so you can swap it, fake it in tests, and keep its details out of your business logic.

The **fake gateway** (`src/fake-gateway/`) is a pretend Stripe that runs inside the app: payment intents, test cards (`4242 4242 4242 4242` succeeds, `4000 0000 0000 0002` is declined, `...0077` goes through "processing" first), signed webhooks, and a **chaos mode** (`FAKE_GATEWAY_CHAOS=true`) that delivers every webhook twice, late and out of order. That lets the whole payment flow run offline and be tested against the worst real-world behaviour.

Starting a payment (`startPayment`)

```
const { payment, created } = await withTransaction(async (trx) => {
  const booking = await trx.selectFrom('bookings').select([... , sql<boolean>`expires_at <=
now()``.as('lapsed')])
    .where('id', '=', bookingId).forUpdate().executeTakeFirst();
  if (!booking || (booking.userId !== user.id && user.role !== 'admin')) throw
  notFound('Booking');
  if (booking.status !== 'pending') throw conflict('BOOKING_NOT_PENDING', '...');
  if (booking.lapsed) throw conflict('HOLD_EXPIRED', 'The seat hold has expired; please book
  again');

  const open = await trx.selectFrom('payments').selectAll()
    .where('bookingId', '=', bookingId).where('status', 'in', ['requires_payment',
    'processing'])
    .executeTakeFirst();
  if (open) return { payment: open, created: false }; // already paying: return the same
  one

  const inserted = await trx.insertInto('payments').values({ bookingId, provider:
  provider.name,
    amountCents: booking.totalCents, currency: booking.currency
  }).returningAll().executeTakeFirstOrThrow();
  return { payment: inserted, created: true };
});

// Network call OUTSIDE the transaction. Idempotency key = our payment id.
const intent = await provider.createPayment({ ... }, `payment-${payment.id}`);
```

Key ideas:

- **One open payment per booking.** Calling twice returns the same payment, so a double-click can't create two charges.
- **The provider call is outside the transaction** (never hold locks while waiting on the network).
- **Idempotency key to the provider.** If our server crashes after the provider created the intent but before we saved its id, the retry sends the same key and the provider returns the *same* intent instead of creating a second one.

Receiving webhooks: verify, store once, answer fast (`ingestWebhook`)

The webhook URL (`POST /api/v1/webhooks/:provider`) is public. Anyone could POST "payment succeeded" to it. So:

1. Verify the signature (`signature.ts`). The provider signs each webhook: `header = t=<timestamp>,v1=<HMAC-SHA256(secret, "<timestamp>.<raw body>")>` . Only the provider and we know the secret.

```
if (Math.abs(now / 1000 - timestamp) > toleranceSeconds) {
  throw new WebhookSignatureError('timestamp outside the tolerance window (possible replay)');
}
const expected = createHmac('sha256', secret).update(`${timestamp}.`).update(payload).digest();
const valid = signatures.some((sig) => {
  const given = Buffer.from(sig, 'hex');
  return given.length === expected.length && timingSafeEqual(given, expected);
});
if (!valid) throw new WebhookSignatureError('signature mismatch');
```

- It verifies the **raw bytes** of the body. If you parsed the JSON and re-serialized it, spacing or key order could change and the signature would not match. That's why the webhook route has its own content parser that keeps the body as a `Buffer` :

```
const webhookRoutes: FastifyPluginAsyncZod = async (app) => {
  app.removeAllContentTypeParsers();
  app.addContentTypeParser('*', { parseAs: 'buffer' }, (_req, body, done) => done(null, body));
  app.post('/webhooks/:provider', { ... }, async (req) => { ... });
};
```

(Fastify plugins are *encapsulated*: this parser change affects only routes inside this plugin.) - The **timestamp** is signed and must be within 5 minutes, so a captured webhook can't be replayed later. - `timingSafeEqual` compares in **constant time**. A normal `===` stops at the first different character, and an attacker measuring response times could guess a signature byte by byte.

2. Store it once. Insert into `webhook_events` whose primary key is `(provider, event_id)` , with `ON CONFLICT DO NOTHING` . A duplicate delivery inserts nothing and is acknowledged and ignored. In the same transaction, `enqueue` a `process-webhook` job.

3. Answer 200 immediately. Providers retry if you're slow, so the real work happens in the job.

Reconciling: don't trust the message, ask for the truth (`reconcilePayment`)

Webhooks arrive **at least once, in any order, sometimes late**. "succeeded" can arrive before "processing". If you acted on each message's content, a late "processing" could overwrite "succeeded". So the job **ignores the webhook's content** and asks the provider for the payment's **current** state:

```
const remote = await provider.retrievePayment(providerPaymentId);
// ...find our payment row...
if (remote.status === 'succeeded' && remote.refundedCents >= remote.amountCents)
```

```

    return completeRefund(payment.id);

    switch (remote.status) {
      case 'succeeded':      return recordSuccess(payment.id);
      case 'processing':     /* mark processing (only if still requires_payment) */
      case 'canceled':       /* mark canceled */
      case 'requires_payment': /* e.g. card declined: store lastError, the buyer can retry */
    }
  }

```

However many times and in whatever order webhooks arrive, every run moves our records toward the same final state. That property is called **convergence**, and it's what makes the system robust.

Recording success, or refunding automatically (`recordSuccess`)

```

return await withTransaction(async (trx) => {
  // seats → booking (inside confirmBookingInTx) → payment: the global lock order
  const outcome = await confirmBookingInTx(trx, before.bookingId, { lateAllowed: true });
  const payment = await trx.selectFrom('payments').selectAll().where('id', '=', paymentId)
    .forUpdate().executeTakeFirstOrThrow();
  if (payment.status === 'succeeded' || payment.status === 'refunded') throw new
    AlreadyRecorded();

  await trx.updateTable('payments').set({ status: 'succeeded', succeededAt: sql`now()` ,
    lastError: null })
    .where('id', '=', paymentId).execute();

  switch (outcome.kind) {
    case 'confirmed':      return { outcome: outcome.late ? 'confirmed_late' : 'confirmed',
    ... };
    case 'already_confirmed': await createRefund(trx, payment, 'duplicate_payment'); ...
    case 'event_unavailable': await createRefund(trx, payment, 'event_cancelled' or
    'hold_expired'); ...
    case 'seats_lost':
    case 'hold_expired':
    case 'not_confirmable': await createRefund(trx, payment, 'hold_expired'); ...
  }
});

```

The payment is recorded and the booking confirmed in **one transaction**. If the booking can't be fulfilled any more (the hold lapsed and someone else bought the seats, the event was cancelled, or it was already paid by another payment), the same transaction records the money **and creates a refund**, and a job executes it with the provider. The buyer gets an email saying so. Nobody pays for nothing.

`AlreadyRecorded` is a small trick: if two jobs for the same payment race, the second one notices under the lock and **throws to roll back** everything it did, then reports "already recorded".

Refunds

- `createRefund` inserts a `refunds` row; a partial unique index allows only one live refund per payment, so a second attempt is a no-op. It queues a `refund` job.
- `executeRefund` (the job) calls `provider.refundPayment` with an idempotency key, so a retried job never refunds twice.

- When the provider confirms (again via webhook + reconcile), `completeRefund` marks the payment `refunded`, ends the booking (`endConfirmedBookingInTx` : tickets voided, seats back on sale), and queues a `refund-processed` email.
- Buyers can request a refund (`POST /bookings/:id/refund`) until `REFUND_CUTOFF_HOURS` (24) before the event. Cancelling an event queues `refund-event`, which refunds every paid booking.

Idempotency keys for clients (`src/lib/idempotency.ts`)

A buyer clicks "Hold seats", the network drops, and the browser doesn't know if it worked. Retrying could create a second booking. The solution, borrowed from Stripe: the client sends an `Idempotency-Key: <random id>` header; the browser reuses the same key when retrying.

```
first request      → runs; the response is stored under (user, key)
retry, same body   → the stored response is replayed (header Idempotent-Replayed: true)
concurrent retry   → 409 IDEMPOTENCY_REQUEST_IN_PROGRESS, Retry-After: 1
same key, new body → 422 IDEMPOTENCY_KEY_REUSED
request failed     → the key is deleted, so the client may retry for real
```

How it works: the key is **claimed first** by inserting a row with `ON CONFLICT DO NOTHING` (so two concurrent requests can't both run), together with a SHA-256 hash of the method, route, params and body (to detect the same key used for a different request). After the handler succeeds, its status and body are saved on that row. If a process crashed mid-request, its claim can be taken over after 60 seconds.

Payment lessons you can reuse anywhere you integrate an external system: verify what comes in; store incoming events with a unique key to dedupe; acknowledge fast and process in a job; re-read the source of truth instead of trusting messages; make every step idempotent; and use idempotency keys on every call that moves money.

Tickets and check-in

Issuing tickets

When a booking is confirmed, `issueTickets` inserts one `tickets` row per seat, **in the same transaction**. So tickets exist exactly when the booking is confirmed. A partial unique index (one valid ticket per seat) makes a second valid ticket for the same seat impossible even if some logic went wrong.

Signing tickets (`tickets/signing.ts`)

A QR code must be hard to forge. The code encodes a **signed token**:

```
payload   = version (1 byte) + ticket id (16 bytes) + event id (16 bytes)
signature = Ed25519(payload)
token      = base64url(payload) + "." + base64url(signature)    (~130 characters)
```

```
export function signTicket({ ticketId, eventId }: TicketClaims): string {
  const payload = Buffer.concat([Buffer.from([VERSION]), uuidToBytes(ticketId),
    uuidToBytes(eventId)]);
  const signature = sign(null, payload, privateKey);
  return `${payload.toString('base64url')}.${signature.toString('base64url')}`;
}

export function verifyTicket(token: string): TicketClaims | null {
  const parts = token.trim().split('.');
  if (parts.length !== 2) return null;
  const payload = Buffer.from(parts[0]!, 'base64url');
  const signature = Buffer.from(parts[1]!, 'base64url');
  if (payload.length !== 33 || payload[0] !== VERSION || signature.length !== 64) return null;
  if (!verify(null, payload, publicKey, signature)) return null;
  return { ticketId: bytesToUuid(payload.subarray(1, 17)), eventId:
    bytesToUuid(payload.subarray(17, 33)) };
}
```

Why these choices (from the comments in the file):

- **A signature instead of just the ticket id:** a fake or mistyped code is rejected without a database lookup, and nobody can create valid tickets by guessing ids.
- **Ed25519 (a public/private key pair) instead of HMAC (one shared secret):** door scanners only need the **public** key (`GET /tickets/public-key`) to verify tickets, even offline. If scanners held an HMAC secret, anyone who extracted it from a device could forge tickets.
- **Binary UUIDs instead of JSON:** a shorter token makes a smaller QR code that scans more easily.

`qrcode` turns the token into a PNG image, attached to the confirmation email and returned by `GET /bookings/:id/tickets`.

Check-in: each ticket admits once (`POST /check-in`)

The signature proves a ticket is genuine. Whether it has *already been used* is a separate question for the database:

```
const admitted = await db
  .updateTable('tickets')
  .set({ checkedInAt: sql`now()`, checkedInBy: user.id })
  .where('id', '=', ticket.id)
  .where('status', '=', 'valid')
  .where('checkedInAt', 'is', null)           // only if never checked in
  .returning('checkedInAt')
  .executeTakeFirst();
if (!admitted) { /* 409 TICKET_VOID or ALREADY_CHECKED_IN */ }
```

This is a **conditional update**, a simple and powerful concurrency tool. Two scanners reading the same QR code at the same moment both run this `UPDATE`, but only one can change `checked_in_at` from `NULL`; the other gets zero rows back and answers "already used". No explicit lock needed: the condition is checked atomically by the database.

Posters: uploading files

Organizers can upload an event poster. The interesting part is that **the file never passes through the API server**:

1. `POST /events/:id/poster/upload-url` returns a **presigned POST** for S3 (MinIO locally): a URL plus signed form fields that allow exactly one upload, of an image, up to 10 MB, within a few minutes (`content-length-range` condition in `lib/storage.ts`).
2. The browser uploads the file **directly to S3**.
3. `PUT /events/:id/poster { key }` checks the object exists, sets `poster_status = 'processing'`, and queues a `process-poster` job (answers 202 Accepted).
4. The worker's `media.ts` handler downloads the original, checks it's really an image (and not a "decompression bomb" that expands to billions of pixels), and uses **sharp** to create WebP versions 320, 640 and 1280 px wide, uploaded with "cache forever" headers (the key changes with every upload). Then `poster_status = 'ready'`.

```
const updated = await db.updateTable('events')
  .set({ posterStatus: 'ready', posterVariants: JSON.stringify(variants), posterError: null })
  .where('id', '=', eventId)
  .where('posterKey', '=', key)           // only if this is still the latest upload
  .executeTakeFirst();
```

Again a conditional update: if the organizer uploaded a newer poster while this job ran, the old result is discarded.

Why so many steps? Uploads are big and slow. Streaming them through the API would tie up the server; resizing images is CPU-heavy and would freeze Node's single event loop for every other request. So the heavy work goes to S3 (storage) and the worker (processing), and a non-image file fails with `UnrecoverableError` (no retries, since retrying won't help).

Live seat maps (WebSockets)

When someone holds a seat, everyone watching that event should see it change immediately. With three API servers, the viewer might be connected to server 1 while the hold happened on server 3. Here's how the message gets there.

Step 1: publish after commit (`realtime/seat-updates.ts`)

Every seat mutation uses `RETURNING id, event_id, status, version` and passes the rows to `notifySeatChanges`:

```
export function notifySeatChanges(changes: SeatChange[]): void {
  if (!changes.length) return;
  const byEvent = new Map<string, SeatTuple[]>();
  for (const c of changes) {
    const list = byEvent.get(c.eventId) ?? [];
    list.push([c.id, c.status, c.version]); // compact tuples, not objects
    byEvent.set(c.eventId, list);
  }
  afterCommit(async () => { // never announce a change that rolls back
    const pipeline = redis.pipeline();
    for (const [eventId, seats] of byEvent) pipeline.publish(seatChannel(eventId),
JSON.stringify(seats));
    await pipeline.exec();
  });
}
```

Redis pub/sub: `PUBLISH seats:<eventId> <message>` sends the message to every connection currently subscribed to that channel, on any server.

Step 2: each server's hub fans out (`realtime/hub.ts`)

Each API server has one `LiveSeatHub`. When a browser opens `GET /api/v1/events/:id/live` as a WebSocket, the route calls `hub.join(eventId, socket)`:

```
async join(eventId: string, socket: WebSocket): Promise<boolean> {
  if (this.connections >= MAX_CONNECTIONS) {
    socket.close(1013, 'Server busy; try again'); // 1013 = try again later
    return false;
  }
  // ...track connection, heartbeat and close handler...
  let viewers = this.sockets.get(eventId);
  if (!viewers) {
    viewers = new Set();
    this.sockets.set(eventId, viewers);
    await this.subscriber.subscribe(seatChannel(eventId)); // first viewer of this event here
  }
  viewers.add(socket);
  return true;
}
```

- A server subscribes to an event's channel only while it has viewers for that event.

- A **dedicated Redis connection** is used for subscribing, because a subscribed connection can't run other commands.

When a message arrives, the hub **batches** changes for 100 ms, keeping only the newest version of each seat, then sends one message per event to all its viewers (serialized once, not once per viewer):

```
private onMessage(channel: string, message: string) {
  const eventId = channel.slice('seats:'.length);
  if (!this.sockets.has(eventId)) return;
  let buffer = this.pending.get(eventId);
  if (!buffer) this.pending.set(eventId, (buffer = new Map()));
  for (const tuple of JSON.parse(message) as SeatTuple[]) {
    const previous = buffer.get(tuple[0]);
    if (!previous || previous[2] < tuple[2]) buffer.set(tuple[0], tuple); // newest version wins
  }
  this.flushTimer ??= setTimeout(() => this.flush(), FLUSH_MS);
}
```

Protections built in:

- **Backpressure:** if a slow client has more than 1 MiB of unsent data, it's disconnected (`socket.terminate()`), so one bad phone connection can't eat the server's memory. It will reconnect and reload.
- **Heartbeats:** every 30 s the hub pings each socket; a socket that didn't answer the previous ping is dropped (dead Wi-Fi, sleeping laptops).
- **Connection cap:** 20,000 per server.
- **Graceful shutdown:** on deploy, sockets are closed with code **1001** ("going away"), and the browser reconnects to another server.

Step 3: the client applies updates by version

The most subtle idea: the browser **subscribes first, then loads the snapshot**, and applies an update only if its `version` is newer than what it has. That handles every tricky case:

- a change that happens between loading the map and subscribing: impossible to miss, because you subscribed first;
- the same update twice, or updates out of order: ignored, because the version isn't newer.

This is the next chapter.

The browser page (public/app.js)

`public/` contains a demo page: `index.html` (the structure), `styles.css` (the look) and `app.js` (the behaviour). It is **plain JavaScript with no framework and no build step**, which makes it a great place to see how a front end talks to a back end.

A tiny API client with automatic token refresh

```
async function api(path, { method = 'GET', body, headers = {}, retryAuth = true } = {}) {
  const res = await fetch(`/api/v1${path}`, {
    method,
    credentials: 'same-origin', // send cookies (the refresh cookie)
    headers: {
      ...(body === undefined ? {} : { 'content-type': 'application/json' }),
      ...(state.token ? { authorization: `Bearer ${state.token}` } : {}),
      ...headers,
    },
    body: body === undefined ? undefined : JSON.stringify(body),
  });
  // Access tokens live 15 minutes; on 401, trade the httpOnly refresh cookie for a new one.
  if (res.status === 401 && retryAuth && state.token && (await refreshSession())) {
    return api(path, { method, body, headers, retryAuth: false }); // retry once
  }
  const data = res.status === 204 ? null : await res.json().catch(() => null);
  if (!res.ok) throw new ApiError(res.status, data); // uses the { error: { code, message } }
  } shape
  return data;
}
```

Every call goes through this one function: it adds the access token, and when the server says "401, token expired", it silently calls `/auth/refresh` and retries once. The user never notices tokens expiring.

Live updates: subscribe, then snapshot

```
function connectLive(eventId) {
  const ws = new WebSocket(`${location.protocol === 'https:' ? 'wss' : 'ws'}://${location.host}/api/v1/events/${eventId}/live`);
  state.ws = ws;

  ws.onmessage = async (message) => {
    const data = JSON.parse(message.data);
    if (data.type === 'hello') {
      // Subscribed first, snapshot second: nothing can fall in between.
      await loadSnapshot(eventId, Date.now());
    } else if (data.type === 'seats') {
      if (state.snapshotReady) applyUpdates(data.seats);
      else state.buffered.push(...data.seats); // updates that arrive during loading
    }
  };

  ws.onclose = (e) => {
    // Exponential backoff with jitter: a whole audience reconnecting at once (say, after a
  }
}
```

```

// deploy) shouldn't hit the servers in one synchronized wave.
const delay = Math.min(15_000, 1_000 * 2 ** state.wsAttempt++) * (0.5 + Math.random() / 2);
state.reconnectTimer = setTimeout(() => connectLive(eventId), delay);
};
}

function applyUpdates(tuples) {
  for (const [id, status, version] of tuples) {
    const entry = state.seats.get(id);
    if (!entry || version <= entry.version) continue; // stale or duplicate: ignore
    entry.status = status;
    entry.version = version;
    if (status !== 'available' && state.selected.has(id)) {
      state.selected.delete(id);
      showMessage(`${entry.label} was just taken by someone else.`);
    }
    paint(id, true);
  }
}
}

```

Exponential backoff with jitter: wait 1 s, 2 s, 4 s, 8 s... (capped at 15 s), each multiplied by a random 0.5 to 1. Without the randomness, 10,000 browsers disconnected by a deploy would all reconnect at exactly the same instant.

The seat map is drawn as **SVG**: one `<rect>` per seat at `(x, y)` from the API, coloured by status.

Hold → pay → tickets

1. **Hold:** the page generates `crypto.randomUUID()` as the `Idempotency-Key` and calls `POST /events/:id/bookings`, retrying on network errors *with the same key* (`withNetworkRetry`), so a retry can never create a second booking. A countdown shows the 10-minute hold.
2. **Pay:** `POST /bookings/:id/payment` returns a client secret; the page sends the test card to `/fake-gateway/v1/payment_intents/:id/confirm` (standing in for Stripe.js).
3. **Wait:** the booking becomes confirmed only when the webhook has been processed by the worker, so the page **polls** `GET /bookings/:id` until the status changes (`waitForOutcome`). If it never does, it suggests checking that the worker is running.
4. **Tickets:** `GET /bookings/:id/tickets` returns the QR codes as images.

The server also sends a strict **Content-Security-Policy** header for these pages (in `app.ts`): only scripts from the same site may run, which blocks most injected-script (XSS) attacks.

Staying fast under load

A flash sale sends more requests than the database can handle. This chapter covers how the app stays up. The guiding principle (ADR 0009): **fail fast and shed load; never queue without limit.**

Two caches (`src/lib/cache.ts`)

1. MicroCache: in-process, 1 second, for hot data that changes constantly (seat maps, availability counts).

```
async get(key: string, load: () => Promise<string>) {
  const hit = this.entries.get(key);
  if (hit && hit.expires > Date.now()) return hit;           // fresh: return it
  const running = this.inflight.get(key);                   // someone is loading it:
  if (running) return running;
  share!
  const loading = load().then((body) => { /* store with etag, expires = now + ttl */ })
    .finally(() => this.inflight.delete(key));
  this.inflight.set(key, loading);
  return loading;
}
```

The second `if` is **single-flight**: when the entry expires and 1,000 requests arrive at once, only **one** runs the database query; the other 999 wait for that same promise. So seat-map load is at most **one query per second per server**, no matter how many people watch. Staleness of up to 1 second is fine because WebSockets deliver every change anyway.

Responses also get an **ETag** (a fingerprint of the body). A browser that already has that version sends `If-None-Match` and gets `304 Not Modified` with no body (`lib/http-cache.ts`).

2. Redis read-through with generation counters: for data read a lot but changed rarely (event pages, public listings).

The hard part of caching is *invalidation* (removing stale entries). The naive way, "delete the cache key after a write", has a race:

```
reader: cache miss → SELECT (old data) ..... SET cache = old data ← stale!
writer:                                     UPDATE, COMMIT, DELETE cache key
```

This project puts a **generation number** in the key: `cache:event:42:<gen>`. A write **increments** the generation after commit (`invalidate(...)` uses `afterCommit`). The slow reader above stores its stale data under the *old* generation, which nobody reads any more. Problem solved without any locks.

```
export async function readThrough(cache, baseKey, generations, ttlSeconds, load) {
  let key: string | null;
  try {
    const gens = generations.length ? await redis.mget(...generations) : [];
    key = `cache:${baseKey}:${gens.map((g) => g ?? '0').join('.')}`;
  }
```

```

const hit = await redis.get(key);
if (hit !== null) return { body: hit, etag: etagOf(hit) };
} catch (err) {
  key = null; // Redis down: just read the database (slower, never
wrong)
}
const body = await load();
if (key) redis.set(key, body, 'EX', ttlSeconds).catch(() => {});
return { body, etag: etagOf(body) };
}

```

The cache is never used to **decide** anything important. Holding a seat always checks Postgres. Caches only serve reads.

Rate limiting: the token bucket (`src/lib/rate-limit.ts`)

Picture a bucket holding up to `capacity` tokens, refilled at `refillPerSec`. Each request takes one token. Empty bucket → 429 Too Many Requests with a `Retry-After` header. This allows short bursts but limits the sustained rate.

The bucket lives in **Redis**, so all three API servers share it (in-memory limits would give each server its own allowance, tripling the real limit). The read-modify-write runs as a **Lua script**, which Redis executes atomically, so two concurrent requests can't both spend the last token. It uses Redis's own clock (`redis.call('TIME')`) because app servers' clocks may differ.

Limits in the app:

Limit	Where
120 burst / 30 per second per IP, for the whole API	<code>app.ts</code> hook
holds: 10 burst, then 1 every 2 s per user	<code>bookings/routes.ts</code>
login: per IP and per IP+email	<code>auth/routes.ts</code>
signup, password reset, password change	<code>auth/routes.ts</code>

Like the other Redis features, it **fails open**: if Redis is down, requests are allowed.

Overload: bounded waiting and cheap rejections

- **Cheap rejections first**: rate limits (429) and the claim gate (409) turn away work *before* it takes a database connection.
- **Bounded waiting**: a request waits at most 5 s for a pool connection, then gets 503 + `Retry-After`. Postgres kills statements after 15 s.
- **Pool arithmetic**: 3 API servers × 20 connections + worker 10 + a listener must stay below Postgres's `max_connections` (set to 200 in `docker-compose.yml`). Bigger pools don't help: more connections make Postgres *slower* under contention.
- **Nginx must not amplify**. A real bug they found under load (commented in `deploy/nginx/local.conf`): Nginx was set to retry on 503. It counted every deliberate 503 as a server failure, marked all three servers dead, and answered **everything** with 502

for seconds. The fix: `proxy_next_upstream error timeout;` (retry only when a server can't be reached).

- **Least connections:** `least_conn` sends each request to the server with the fewest in-flight requests, better than round-robin when some requests (holds) are much slower than others (cached reads).

The result measured in the load test (k6, 1,000 buyers per second on a laptop): 0 errors, holds at 1.4 s at the 95th percentile, and the invariant check found no double sale.

Checking the rules after the storm (`src/lib/invariants.ts`)

Each invariant is a SQL query that returns the rows breaking a rule, so a healthy database returns nothing: "a seat belongs to more than one active booking", "a booked seat whose booking isn't confirmed", "a succeeded payment that neither confirmed its booking nor was refunded", and more. They run after tests, after load tests (`npm run check:invariants`) and on demand (`GET /admin/invariants`). This is how "no seat was sold twice" is **verified rather than assumed**.

Logs, metrics, health checks, shutdown

When something breaks in production you can't attach a debugger. You need to *observe* the system.

Structured logs (`src/lib/logger.ts`)

The app uses **pino** and writes one **JSON** line per event:

```
{
  "level": "info",
  "time": "2026-09-30T12:00:00.000Z",
  "instance": "api-2:1",
  "requestId": "4f1c...",
  "userId": "9ab...",
  "msg": "request completed",
  "res": {
    "statusCode": 201,
    "responseTime": 38
  }
}
```

- JSON lines can be searched and filtered by log tools (`requestId = "4f1c..."`).
- Every line has the **instance** and, for requests, the **request id**. The outbox stores that request id with each job, and the worker logs it, so **one id follows a click through Nginx, the API and every background job it caused**.
- **Redaction**: `authorization` headers, cookies, passwords, tokens and client secrets are replaced with `[redacted]` automatically.
- `LOG_PRETTY=true npm run dev` prints human-friendly lines during development.

Metrics (`src/lib/metrics.ts`)

Metrics are numbers over time, scraped by **Prometheus** from `GET /metrics` and graphed by **Grafana** (a dashboard is included in `deploy/monitoring/`).

- `http_request_duration_seconds{method, route, status_code}` : a histogram, by route *template* (`/events/:id` , not every real id, which would create millions of series);
- `http_requests_in_flight` ;
- `db_pool_connections{state}` (a rising `waiting` count means the database is the bottleneck);
- `cache_requests_total{cache, result}` (hit / miss / coalesced);
- `booking_hold_attempts_total{strategy, outcome}` ;
- `job_duration_seconds` , `queue_jobs{queue, state}` , `outbox_unpublished` (a growing age means the relay is stuck);
- `websocket_connections` , plus Node's own (event-loop lag, memory, GC).

Health checks (`src/modules/health/`)

Two endpoints, deliberately different:

- `GET /health/live` ("liveness"): *can the process answer at all?* No dependency checks. If the database is down, restarting the app won't fix it, and restarting every container in a loop would make things worse.

- `GET /health/ready` ("readiness"): *should traffic come here?* Checks the database (`SELECT 1`) and Redis (`PING`) with 1-second timeouts, and returns 503 while shutting down. Docker's `HEALTHCHECK` and the rollout script use this.

Graceful shutdown (again, now you can see why)

On `SIGTERM`: `lifecycle.beginShutdown()` makes `/health/ready` return 503 → the load balancer stops sending new requests → after `SHUTDOWN_DRAIN_MS` the server stops listening, finishes in-flight requests, closes WebSockets with code 1001 (browsers reconnect elsewhere), then closes the database pool and Redis. The worker similarly stops the relay, waits for running jobs to finish, and exits.

Docker, Nginx, CI and deploying

The Dockerfile: one image, three roles

```
FROM node:24-slim AS base
WORKDIR /app

FROM base AS deps                                # production dependencies only
COPY package.json package-lock.json ./
RUN npm ci --omit=dev

FROM base AS build                                # compile TypeScript
COPY package.json package-lock.json ./
RUN npm ci
COPY tsconfig.json tsconfig.build.json ./
COPY src ./src
RUN npm run build

FROM base AS runtime                              # the final, small image
ENV NODE_ENV=production
RUN apt-get update && apt-get install -y --no-install-recommends tini && rm -rf
/var/lib/apt/lists/*
COPY --from=deps /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
COPY package.json ./
COPY public ./public
USER node                                          # don't run as root
EXPOSE 3000
HEALTHCHECK --interval=10s --timeout=3s --start-period=20s --retries=3 CMD [...fetch
/health/ready...]
ENTRYPOINT ["/usr/bin/tini", "--"]
CMD ["node", "dist/server.js"]
```

- **Multi-stage build:** TypeScript and dev tools exist only in the `build` stage; the final image has just compiled JavaScript and production dependencies. Smaller and safer.
- **One image, three roles:** `node dist/server.js` (API, the default), `node dist/worker.js` (worker), `node dist/db/migrate.js latest` (migrations). Compose chooses by overriding the command.
- **USER node** : if an attacker breaks in, they're not root.
- **tini as PID 1**: inside a container, process #1 doesn't get default signal handling. tini forwards `SIGTERM` to Node so graceful shutdown really runs, and cleans up zombie processes.

docker-compose.yml: the whole system with one command

`docker compose up -d --build` starts: **postgres**, **redis** (with `noeviction`, because BullMQ's keys must never be evicted), **minio** (S3-compatible file storage), **mailpit** (catches outgoing emails so you can view them at `localhost:8025`), a one-shot **migrate** container, **3 API replicas**, the **worker**, and **nginx** on port 8080. An optional `monitoring` profile adds Prometheus and Grafana. Services wait for each other with health checks (`depends_on: condition: service_healthy`).

The settings go in as environment variables, the same `config.ts` schema validates them, and development defaults are used unless you override them.

Nginx

In front of the API replicas (see `deploy/nginx/`):

- `upstream api { least_conn; ... }`: load balancing;
- WebSocket support (`Upgrade` / `Connection` headers, 1-hour timeouts);
- `X-Request-Id $request_id` so Nginx's logs and the app's logs share ids;
- `X-Forwarded-For` so the app knows the real client IP (and the app trusts that header **only** from Nginx's address: `TRUST_PROXY`, otherwise clients could fake their IP to dodge rate limits);
- in production: HTTPS with Let's Encrypt certificates, an edge rate limit, and `/metrics` blocked from the public.

CI: GitHub Actions (`.github/workflows/ci.yml`)

On every push and pull request, GitHub runs:

1. **checks**: `npm ci`, lint (ESLint), format check (Prettier), typecheck (`tsc --noEmit`), build;
2. **test**: the full test suite against real Postgres, Redis, MinIO and `stripe-mock` (Stripe's official fake API) started as service containers;
3. **docker**: builds the image (and on `main`, pushes it to GitHub's container registry tagged with the commit SHA);
4. **smoke**: starts the entire stack with Compose, runs a real headless-browser test of the seat map, a k6 load test, a zero-downtime rollout under load, and the invariant check.

Deploying (`deploy.yml`, `deploy/rollout.sh`, `docs/DEPLOY.md`)

Production is one VPS running Docker Compose. A deploy (run manually from GitHub Actions with a commit SHA):

1. pulls the image CI already built for that commit (servers never build);
2. runs **migrations first**. Migrations only ever *add* things (new columns, new tables); removals come a release later ("expand, then contract"), so the old code keeps working during the deploy and rolling back is just deploying the previous image;
3. runs `deploy/rollout.sh` for the API: start new replicas next to the old ones, wait until healthy, point Nginx at the new ones (graceful reload), stop the old ones (they drain), then point Nginx back at the service name;
4. updates the worker and the rest.

Why a script instead of `docker compose up -d`? They measured it under 65 requests/second: plain `up -d` failed 336 of 2,927 requests, replacing replicas and relying on DNS failed 3, the rollout script failed **0**. CI repeats that check on every push.

Also important for production (`config.ts` and `DEPLOY.md`): secrets live in a `chmod 600 .env.production` file on the server, never in git; the app refuses to start with development secrets; only ports 80 and 443 are open.

Tests

Tests live in `test/` and run with **Vitest** (`npm test`). `vitest.config.ts` defines two groups:

- **unit** (`test/unit/`): pure functions, no database: seat layout generation, pagination cursors, webhook signatures, ticket signing, the Stripe provider against `stripe-mock`. Fast; run with `npm run test:unit`.
- **api** (`test/api/`): real HTTP-level tests against a real Postgres and Redis (a separate test database, wiped before each test). Files run one at a time because they share the database.

`test/helpers.ts` provides the setup every API test file uses:

```
export function useApp() {
  const ctx = {} as { app: FastifyInstance };
  beforeAll(async () => {
    ctx.app = await buildApp({ logger: false }); // the real app, no real port
    await ctx.app.ready();
    configureDelivery({ mode: 'sync', deliverer: async (body, headers) =>
      (await ctx.app.inject({ method: 'POST', url: '/api/v1/webhooks/fake', payload: body,
        headers })).statusCode });
  });
  beforeEach(async () => { await resetState(); }); // truncate every table, flush Redis
  afterAll(async () => { await ctx.app.close(); /* close db, redis, queues */ });
  return ctx;
}
```

- `app.inject({ method, url, headers, payload })` sends a fake HTTP request straight into Fastify, no network needed. That's why `buildApp` is a function.
- The fake payment gateway delivers its webhooks synchronously into the same app, so a test knows the webhook has landed when the payment call returns.
- Helpers like `createUser`, `createVenue`, `createEvent`, `publish`, `payFor` keep each test short.

The real concurrency test from `test/api/bookings.test.ts` reads like a story. It runs once for each correct strategy:

```
describe.each<HoldStrategy>(['pessimistic', 'optimistic', 'serializable'])('concurrency with the %s strategy',
  (strategy) => {
    it('30 buyers race for one seat: exactly one wins, the rest get a clean 409', async () => {
      // Gate off, so every attempt really reaches the database transaction.
      const app = await buildApp({ logger: false }, { booking: { strategy, claimGate: false } });
      try {
        const buyers = await Promise.all(Array.from({ length: 30 }, () =>
          createUser('attendee')));
        const results = await Promise.all(buyers.map((b) => hold(b, [seatIds[0]!], eventId,
          app)));
        const codes = results.map((r) => r.statusCode);
        expect(codes.filter((c) => c === 201)).toHaveLength(1);
        expect(codes.filter((c) => c === 409)).toHaveLength(29);
      } catch {}
    });
  })
```

```

// ...and the database agrees: exactly one pending booking holds that seat
const holders = await db.selectFrom('bookingItems as bi')
  .innerJoin('bookings as b', 'b.id', 'bi.bookingId')
  .where('bi.eventSeatId', '=', seatIds[0]!).where('b.status', '=', 'pending')
  .select((eb) => eb.fn.countAll<number>().as('n')).executeTakeFirstOrThrow();
expect(holders.n).toBe(1);
} finally {
  await app.close();
}
});
},
);

```

`Promise.all` fires all 30 requests at once, which is how you test a race. The test checks both the HTTP answers *and* the database.

Other testing tools in `scripts/`: `race-test.ts` (200 concurrent holds per strategy), `loadtest/flash-sale.js` (k6), `e2e-seatmap.ts` (Playwright: a buyer and a watcher in two headless browsers), `check-invariants.ts`, and `seed.ts` (creates ~1.5 million rows of realistic demo data; every seeded user's password is `password123`).

Lesson: test at the level where bugs actually happen. For a backend, that's usually "send a real HTTP request to the real app with a real database and check the response and the database". Mocks everywhere tend to test your mocks.

Trace one purchase from start to finish

Now let's connect everything. Here is what happens, file by file, when a buyer buys two seats. Try to picture each step; if one is unclear, go back to its chapter.

1. Page load. The browser loads `index.html` and `app.js` (served by `@fastify/static`, with a Content-Security-Policy). `app.js` calls `GET /api/v1/events` → `events/routes.ts` → `readThrough` (Redis cache keyed by the event-lists generation) → on a miss, `listEvents` (keyset pagination) → JSON.

2. Log in. `POST /auth/login` → rate limits (`enforce`) → look up the user → `verifyPassword` (Argon2id) → `startSession` inserts `sessions` + `refresh_tokens` → the response carries the **access token** (JSON) and sets the **refresh cookie** (`httpOnly`).

3. Open the seat map. The page opens a WebSocket to `/events/:id/live` → `hub.join` subscribes this server to `seats:<eventId>` in Redis → `hello` → the page loads `GET /events/:id/seats` → `seatMapCache` (MicroCache, 1 s, single-flight) → `buildSeatMap` (uses `acquirableSql`, so lapsed holds show as free) → drawn as SVG.

4. Hold two seats. `POST /events/:id/bookings` with an `Idempotency-Key`:

- Nginx → a replica (`least_conn`), request id assigned;
- `onRequest` hooks: metrics, request context, per-IP token bucket;
- `requireAuth`: verify the JWT, check the Redis denylist;
- Zod validates `seatIds`;
- per-user hold limit; `withIdempotency` claims the key;
- `holdSeats`: `loadSaleableEvent` → `checkUserLimits` → **claim gate** (Redis Lua `SET NX`) → **transaction**: `SELECT ... FOR UPDATE SKIP LOCKED (seats, ascending id)` → `INSERT bookings (pending, expires_at = now() + 10 min)` → `enqueue expire-booking (outbox row)` → `UPDATE event_seats to held (version + 1)` → `INSERT booking_items` → **COMMIT**;
- after commit: `PUBLISH seats:<eventId>` (live update) and cache generation bumps;
- claims released; the idempotency row stores the response; `201 Created`.

5. Everyone else sees it. Every API replica with viewers of this event receives the Redis message → the hub batches for 100 ms → each viewer's browser gets `[[seatId, "held", version], ...]` → `applyUpdates` paints those seats as held (only if the version is newer).

6. Meanwhile, in the worker. The database trigger NOTIFYS → the `OutboxRelay` wakes (or polls) → `publishOutboxBatch` moves the expire-booking row into BullMQ as a **delayed** job with a fixed id, due in 10 minutes.

7. Pay. `POST /bookings/:id/payment` → `startPayment` (lock booking, reuse or insert a `payments` row) → outside the transaction, `provider.createPayment(..., 'payment-<id>')` → client secret → the browser pays at the provider.

8. Webhook. The provider POSTs a signed webhook to `/api/v1/webhooks/fake` → raw-body parser → `verifyWebhookSignature` (HMAC, timestamp window, constant-time compare) → `INSERT webhook_events ON CONFLICT DO NOTHING` + outbox row for `process-webhook` → 200 in milliseconds.

9. Confirm. Worker: `processWebhook` → `reconcilePayment` → `provider.retrievePayment` says `succeeded` → `recordSuccess` → one transaction: `confirmBookingInTx` (lock seats → booking; seats booked; booking confirmed; issueTickets; outbox row for the booking-confirmed email) → lock payment → payment `succeeded` → COMMIT → after commit, live update: seats turn "sold" on every viewer's map.

10. Email. Relay → `email` queue → `bookingConfirmed` handler → `sendOnce('booking-confirmed', bookingId, ...)` → loads the booking, signs each ticket with Ed25519, renders QR PNGs, sends via SMTP (Mailpit locally) → `notifications` row marked `sent`.

11. The page. `waitForOutcome` polls `GET /bookings/:id` until `confirmed` → `GET /bookings/:id/tickets` → QR codes shown.

12. Ten minutes later. The delayed `expire-booking` job runs → `expireBooking` locks the booking, sees it is `confirmed` → does nothing. (Idempotent handlers make "late" or "unnecessary" jobs harmless.)

13. At the door. Staff scan the QR → `POST /check-in` → `verifyTicket` (signature) → conditional `UPDATE tickets SET checked_in_at = now() WHERE checked_in_at IS NULL` → admitted exactly once.

Throughout, one **request id** ties together the Nginx log, the API log lines, and the worker's logs for every job this purchase caused, and metrics (`booking_hold_attempts_total` , `job_duration_seconds` , `http_request_duration_seconds`) record it all.

Exercise. Run the stack (`docker compose up -d --build` , then seed as in the README), buy a ticket at `http://localhost:8080` , copy the `x-request-id` from the browser's network tab, and run `docker compose logs api worker nginx | grep <id>` . Then look at the rows in `bookings` , `outbox` , `payments` , `webhook_events` , `tickets` and `notifications` . If you can explain each step and why it is safe to repeat, you understand the system. (`docs/STUDY-GUIDE.md` has this "capstone" and many more experiments.)

How to build your own project

You've now seen a big project from the inside. Here's how to use that to build your own.

The skeleton every backend shares

Almost every backend you build will have these parts. You've seen each one in this project:

Part	In ticket-mng	What to do in yours
Config, validated at startup	<code>src/config.ts</code>	Zod (or similar) schema over env vars; crash on bad config
Entry point + graceful shutdown	<code>src/server.ts</code>	listen; handle SIGTERM; close resources
App factory	<code>src/app.ts</code> (<code>buildApp</code>)	a function that builds the app (tests use it)
Feature modules	<code>src/modules/<feature>/</code>	<code>routes.ts</code> (HTTP) + <code>service.ts</code> (rules)
Database access	<code>src/db/</code>	a pool, migrations, a transaction helper
One error format + central handler	<code>src/lib/errors.ts</code>	<code>AppError</code> + one <code>setErrorHandler</code>
Validation at the edge	Zod schemas on each route	validate every input; shape every output
Auth	<code>src/modules/auth/</code>	hashed passwords; tokens or sessions; a guard
Logging	<code>src/lib/logger.ts</code>	structured JSON logs with a request id
Tests	<code>test/</code>	HTTP-level tests with a real database

A step-by-step recipe for a new project

Build in **phases**, like this project did (its README roadmap has 7). Each phase should end with something that works.

1. **Write down the domain.** Nouns become tables (user, venue, event, seat, booking). Verbs become endpoints or state transitions (hold, pay, cancel). Draw the states of anything with a status.
2. **Design the tables first** and put rules in the database: `NOT NULL` , `CHECK` , foreign keys, unique indexes. Write them as migrations.
3. **Skeleton:** config, `buildApp` , one health route, error handler, logger. Get `GET /health` working.
4. **First resource end to end:** one `routes.ts` with create/list/get, Zod schemas, a DTO, and a test using `app.inject` .
5. **Auth:** signup, login, a guard, roles, ownership checks.

6. **The core business action** (for you it might be "place order", "book appointment", "enrol in course"). Ask: *what if two people do this at the same moment?* Use a transaction and either a row lock, a conditional update, or a unique constraint. Write a concurrency test with `Promise.all`.
7. **Slow or later work** goes to a queue. Start simple; add the outbox pattern once jobs must match database changes.
8. **Only then** add caching, rate limits, WebSockets, Docker and CI, when you have the problem they solve.

Questions to ask about every feature

These are the questions this codebase keeps asking. Asking them is what separates a toy project from a real one:

- **What if this runs twice?** (retries, double-clicks, duplicate webhooks) → make it idempotent: unique keys, `ON CONFLICT DO NOTHING`, conditional updates, idempotency keys.
- **What if two of these run at the same time?** → transactions, locks in a consistent order, constraints.
- **What if it crashes halfway?** → transactions; the outbox; state derived from timestamps instead of timers.
- **What if the other service is down or slow?** → timeouts, retries with backoff and jitter, fail open or fail closed deliberately.
- **What can a malicious user send?** → validate everything; check ownership; rate limit; don't leak whether things exist.
- **How will I know it's broken in production?** → logs with request ids, metrics, health checks.
- **How do I prove it works?** → tests at the HTTP level, concurrency tests, invariant checks.

Patterns you can now name (and reuse)

Pattern	One-line meaning	Where you saw it
Routes / services split	HTTP code separate from business rules	every module
DTO	explicit output objects, nothing leaks	<code>toUserDto</code> , <code>toEventDto</code>
Migrations	versioned schema changes, never edited once run	<code>src/db/migrations/</code>
Database constraints as rules	the database enforces invariants	<code>EXCLUDE</code> , <code>CHECK</code> , partial unique indexes
Row locking + SKIP LOCKED	take rows exclusively; losers skip instead of waiting	holds, outbox relay
Optimistic concurrency	"update only if version unchanged"	optimistic strategy

Pattern	One-line meaning	Where you saw it
Conditional update	<code>UPDATE ... WHERE state = expected</code> as an atomic check	check-in, posters
Global lock order	always lock in the same order → no deadlocks	seats → booking → payment
Lazy expiry	compute "expired" from a timestamp at read time	<code>acquirableSql</code>
Transactional outbox	jobs saved in the same transaction as the change	<code>src/jobs/outbox.ts</code>
Idempotent handler	safe to run twice	<code>sendOnce</code> , <code>expireBooking</code>
Dead-letter queue	failed jobs parked for humans	<code>runner.ts</code>
Idempotency key	a retried request replays the original response	<code>lib/idempotency.ts</code>
Reconciliation	re-read the source of truth, don't trust messages	<code>reconcilePayment</code>
Adapter	wrap an external service behind your own interface	payment providers
After-commit effects	notify only after data is saved	<code>afterCommit</code>
Pub/sub fan-out	one publish reaches all servers' clients	<code>realtime/</code>
Versioned updates	clients ignore stale or duplicate updates	<code>seat version</code>
Single-flight cache	concurrent misses share one load	<code>MicroCache</code>
Generation-keyed cache	invalidate by bumping a counter	<code>readThrough</code>
Token bucket	burst + sustained rate limit	<code>rate-limit.ts</code>
Fail fast / load shedding	429 / 409 / 503 early instead of queueing	ADR 0009
Liveness vs readiness	"alive?" vs "should get traffic?"	<code>health/</code>
Graceful shutdown	stop taking work, finish, then exit	<code>server.ts</code> , <code>worker.ts</code>
Expand / contract migrations	only add now, remove later	deploys

Project ideas to practise with (smallest first)

1. **URL shortener:** one table, a unique constraint on the short code, a redirect route, a click counter (race: two concurrent clicks). Adds: validation, errors, tests.
2. **To-do API with users:** signup/login (copy the auth ideas), ownership checks (you can only see your own tasks), cursor pagination.
3. **Library book lending:** books with copies, "borrow" must not lend the same copy twice (row lock or conditional update), due dates with lazy "overdue" status, a daily reminder job.

4. **Appointment booking for a clinic:** time slots that must not overlap (try an `EXCLUDE` constraint like `events_no_venue_overlap`), holds that expire, email confirmations through a queue.
5. **Live auction:** bids must be strictly higher (concurrency!), live updates over WebSockets with Redis pub/sub, an auction that closes at a timestamp.

For each: write the tables first, add one feature at a time, and write the concurrency test for the core action.

How to keep learning from this repository

- Read `docs/STUDY-GUIDE.md` : it goes phase by phase with experiments ("remove `.skipLocked()` and rerun the race test") and self-check questions with answers.
- Read the ADRs in `docs/adr/` : each is one page on *why* a decision was made and what alternatives were rejected. Learning the "why" is what lets you make your own decisions.
- Use git as a time machine: `git log --oneline`, then `git show --stat <commit>` to see what each phase added, and `git checkout <commit>` to see the project at that point. Seeing a project grow is the best way to learn how to grow your own.
- Break things on purpose: set `HOLD_STRATEGY=naive` and run `npm run race` ; stop the worker, buy a ticket, look at the `outbox` table, start the worker and watch the email arrive.

Glossary

Term	Meaning
ADR	Architecture Decision Record: a short document explaining one design decision
API	the set of requests a server accepts
Argon2id	a slow, memory-hard password hashing algorithm
At-least-once delivery	a message/job may be delivered more than once, but never zero times
Backoff (exponential)	waiting longer after each failed retry: 1 s, 2 s, 4 s, ...
Backpressure	slowing or dropping a producer when the consumer can't keep up
BullMQ	a job-queue library on Redis
Cache	a fast temporary copy of data
Claim gate	the Redis pre-check that turns away losing seat holds before Postgres
Concurrency	several things happening at the same time
Constraint	a rule the database enforces (CHECK, UNIQUE, FOREIGN KEY, EXCLUDE)
CSRF	a malicious site making your browser send requests with your cookies; blocked by SameSite cookies
Cursor pagination	fetching "the next N after this item" instead of "skip N"
Deadlock	two transactions each waiting for a lock the other holds
Dead-letter queue	where jobs go after failing all retries
DTO	Data Transfer Object: the exact shape sent to clients
Drain	letting in-flight work finish while accepting no new work
Ed25519	a public-key signature algorithm (used for tickets)
ETag / 304	a response fingerprint; "not modified" answers save bandwidth
Fail open / fail closed	on a dependency failure, allow (open) or deny (closed)
Foreign key	a column that must match an id in another table
Generation counter	a number in cache keys, bumped on writes to invalidate
HMAC	a signature made with a shared secret (used for webhooks and JWTs)
Hook (Fastify)	a function run at a stage of every request
httpOnly cookie	a cookie JavaScript can't read
Idempotent	doing it twice has the same effect as once

Term	Meaning
Idempotency key	a client-chosen id that makes retries safe
Index	a data structure that makes lookups fast
Invariant	a rule that must always be true ("no seat sold twice")
Isolation level	how much concurrent transactions can see of each other
Jitter	randomness added to delays so retries don't synchronise
JWT	JSON Web Token: signed claims, verifiable without a database
Lazy expiry	treating something as expired by checking its timestamp at read time
Liveness / readiness	"is the process alive?" / "should it receive traffic?"
Load balancer	spreads requests across several servers (Nginx here)
Lua script (Redis)	a small program Redis runs atomically
Migration	a versioned change to the database structure
Optimistic locking	no locks; detect conflicts with a version number at write time
ORM	Object-Relational Mapper (like Prisma); this project uses a query builder instead
Outbox	a table of jobs written in the same transaction as the change
Pessimistic locking	lock rows while reading (<code>FOR UPDATE</code>)
Plugin (Fastify)	a reusable group of routes/settings
Pool (connection pool)	a set of open database connections shared by requests
Pub/sub	publish a message to a channel; every subscriber receives it
Race condition	a bug caused by unlucky timing between concurrent operations
Rate limit	a cap on how many requests a client may make
Reconciliation	deriving our state from the source of truth, not from messages
Refresh token	a long-lived token used only to get new access tokens
Replica	one of several identical copies of a service
Reverse proxy	a server in front of your app (Nginx)
Rollback	undoing a transaction's changes
Schema (Zod)	a description of data's shape, used to validate it
Serializable	the strictest isolation level: transactions behave as if one after another
Single-flight	concurrent requests for the same thing share one computation
SKIP LOCKED	skip rows locked by others instead of waiting
SQL injection	an attack where input changes a query; prevented by parameters
Token bucket	a rate-limit algorithm allowing bursts plus a steady rate

Term	Meaning
Transaction	a group of changes that all happen or none do
UUID	a random 128-bit unique id
Webhook	an HTTP request a provider sends you when something happens
WebSocket	a long-lived two-way connection between browser and server
Worker	a process that runs background jobs
XSS	injected scripts running in your page; blocked by CSP and httpOnly cookies

Happy building.